

aelion

aelion.fr

Parcours IA

Deep Learning

01

Introduction



Intelligence artificielle
(IA)

Apprentissage automatique
(machine learning)

Forêts aléatoires

Classification
automatique

Regression

Réseaux de neurones

SVM

Deep learning

Prédire :

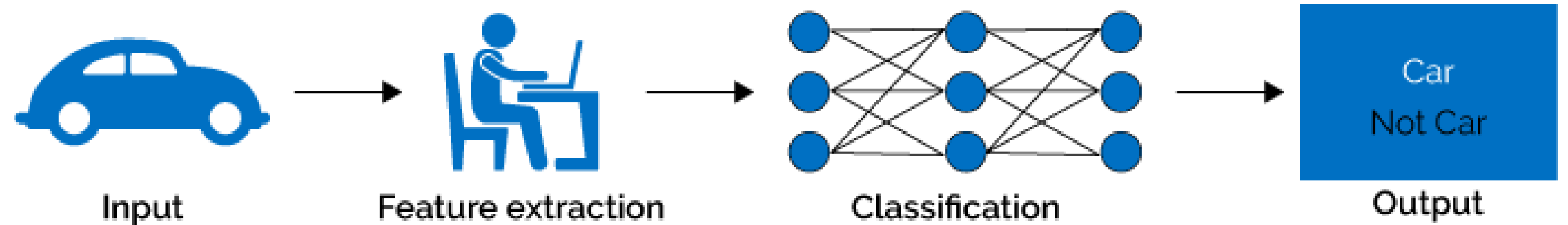
- une classe (valeur discrète)
- la probabilité d'appartenance à une classe

Prédire une valeur
numérique continue

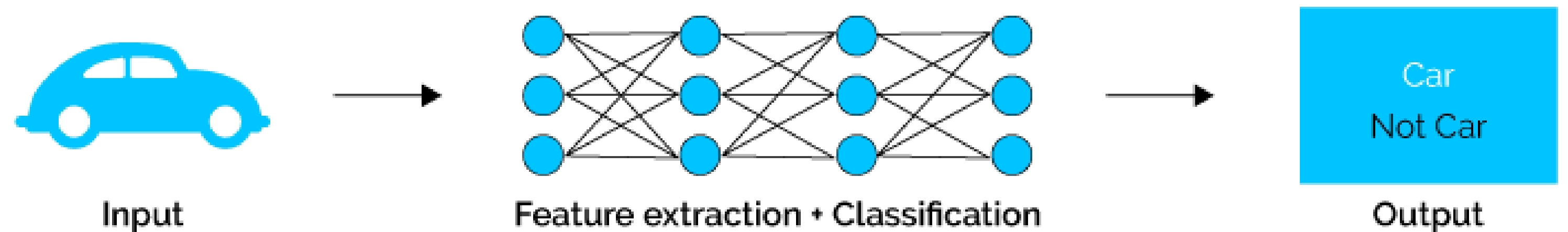
Machine Learning vs Deep Learning

aelion.fr

Machine Learning



Deep Learning



02

MLP



Réseaux de neurones multicouches

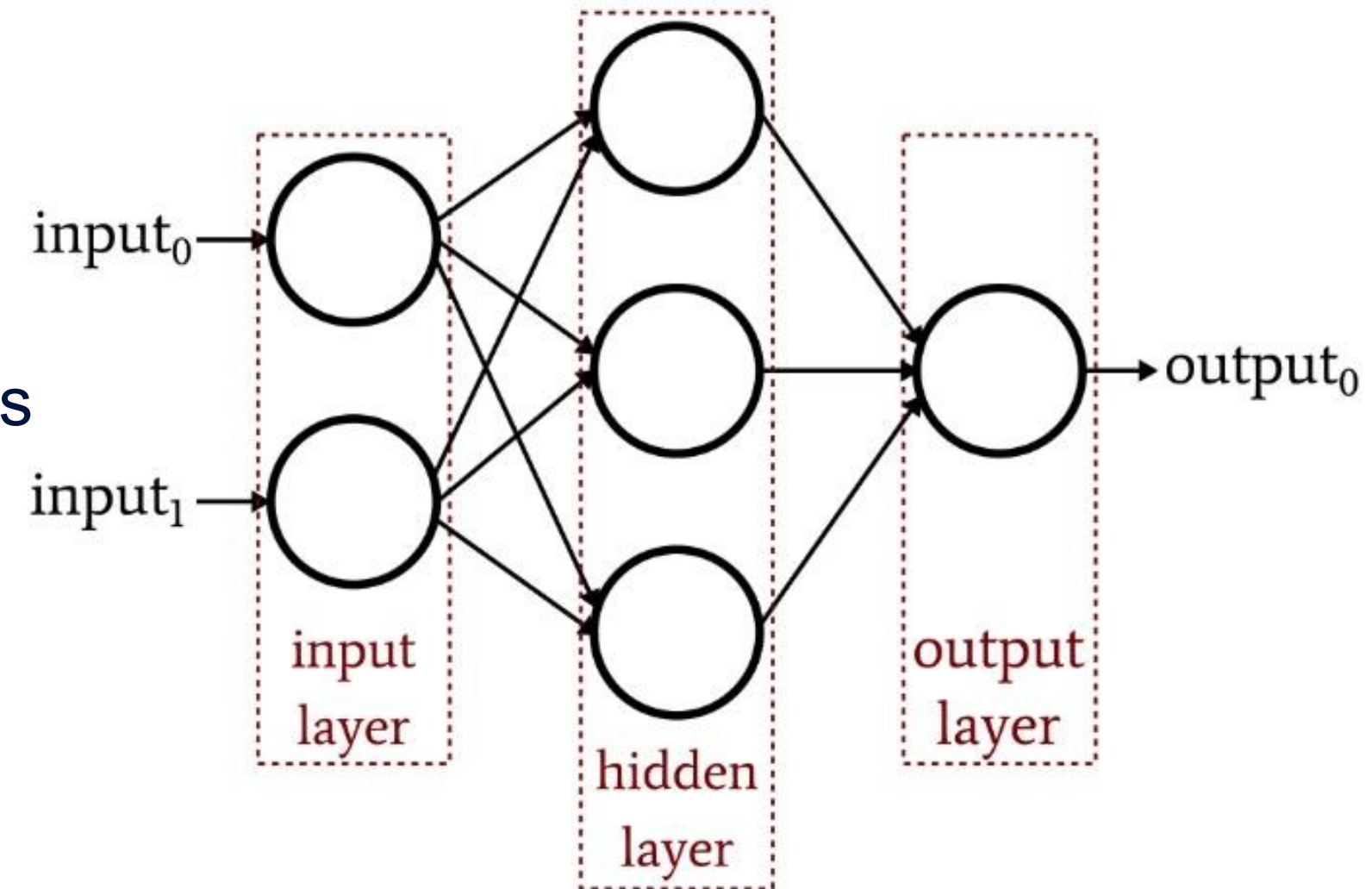
MLP

MLP = Multi Layer Perceptron

Plusieurs couches :

- Une couche « Input »
- Une ou plusieurs couches cachées
- Une couche « Output »

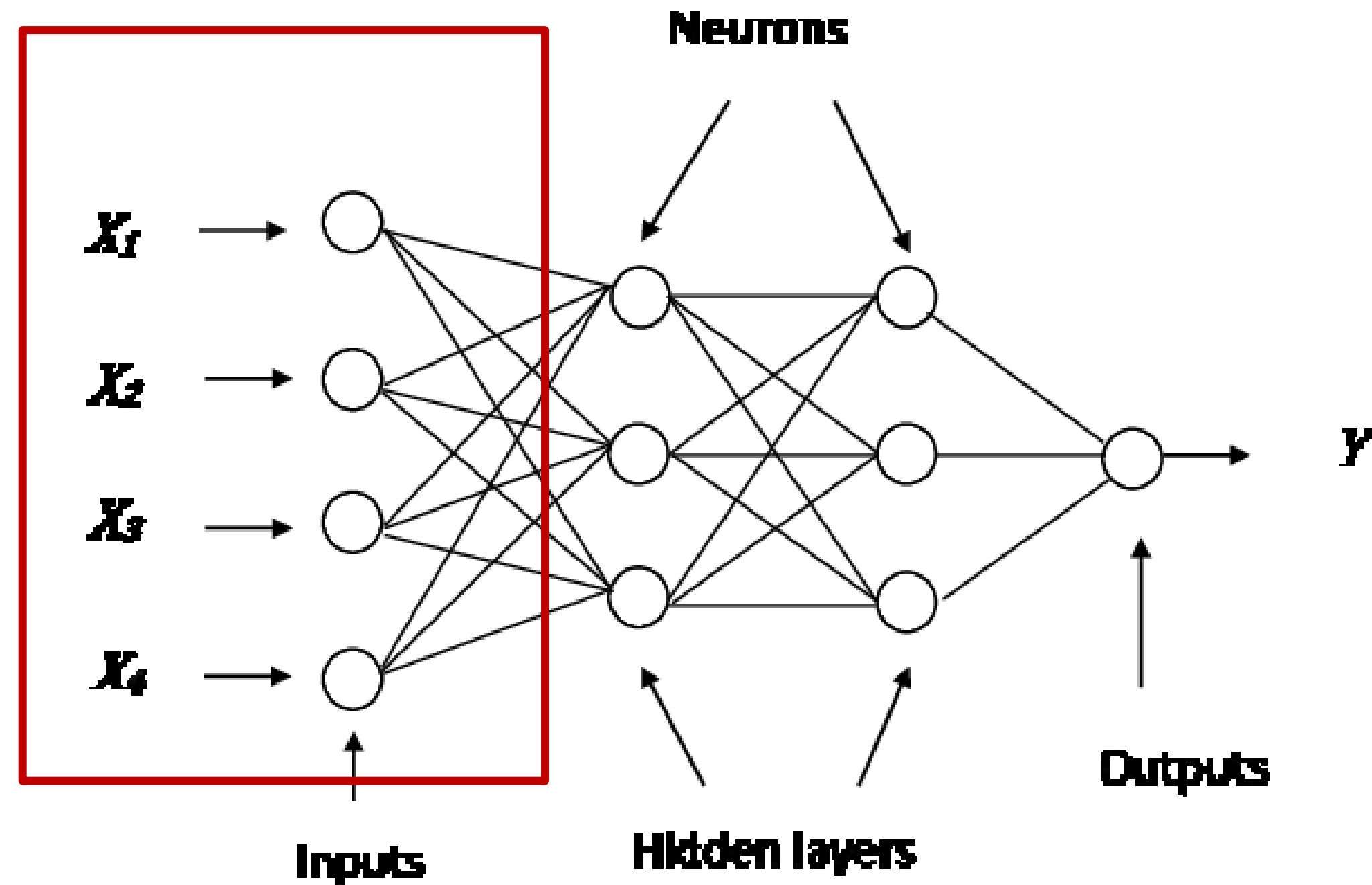
Plusieurs neurones par couche, sauf éventuellement l'output



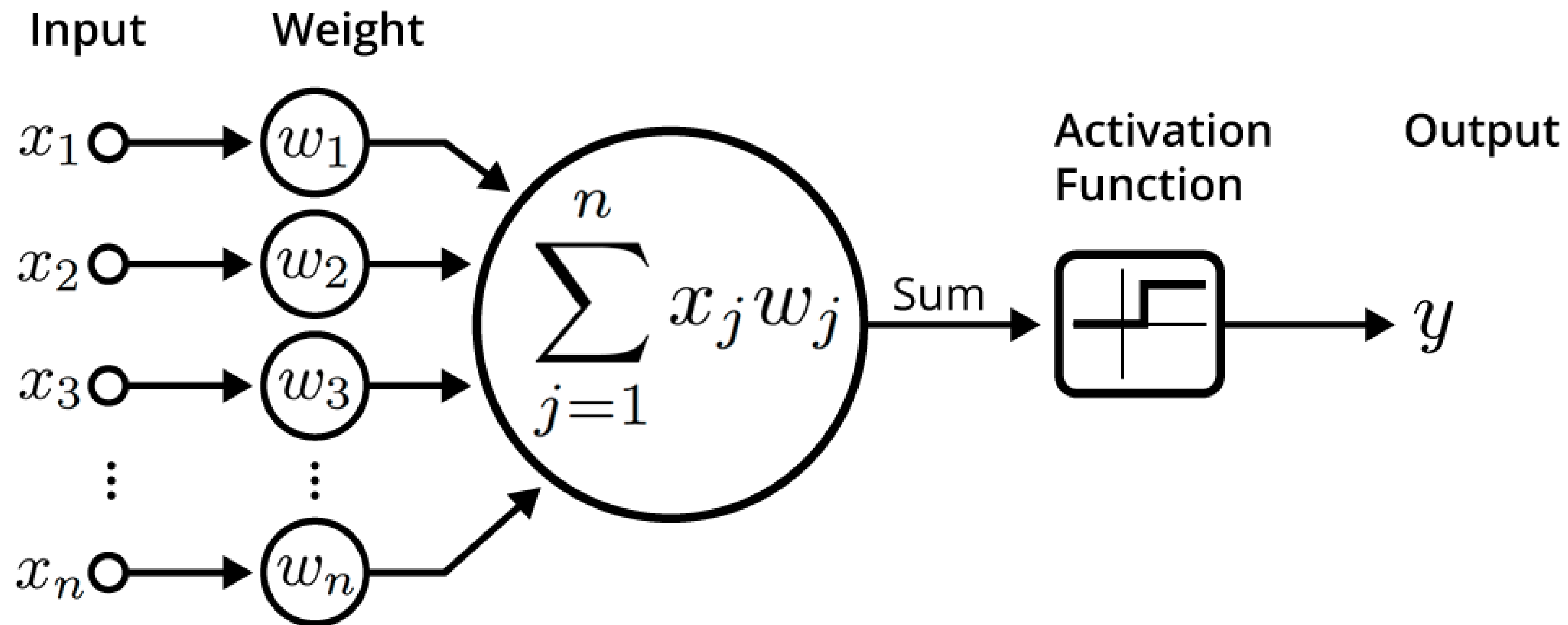
Input Layer

1 neurone = 1 feature
Que des features numériques

Même échelle : normalisée et standardisée

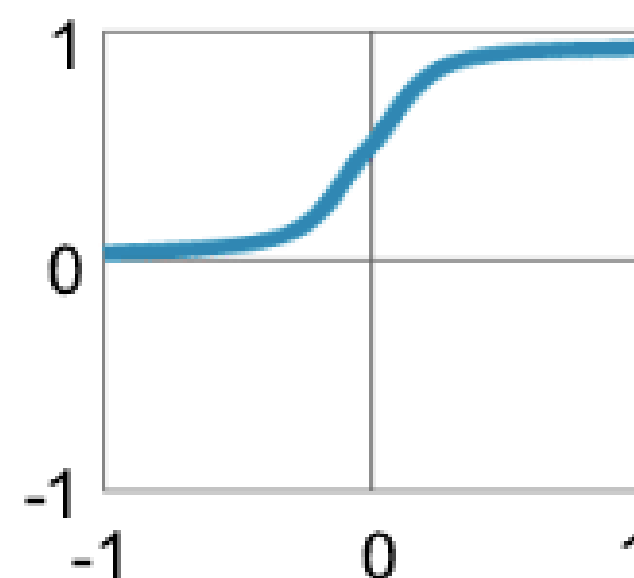


Neurone



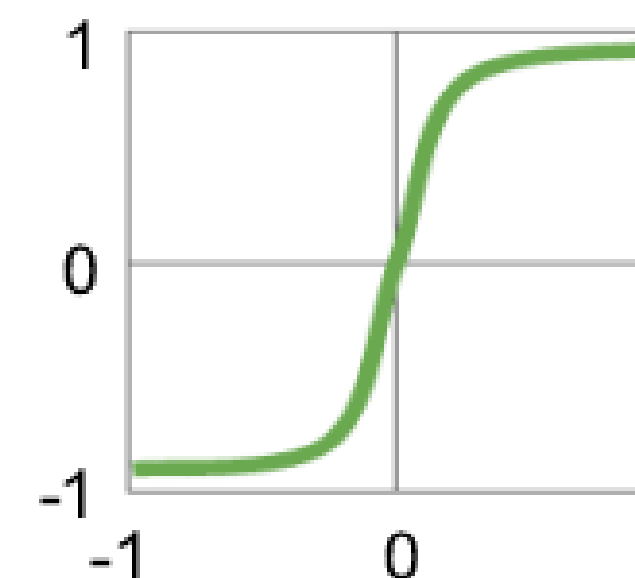
Traditional Non-Linear Activation Functions

Sigmoid



$$y = 1 / (1 + e^{-x})$$

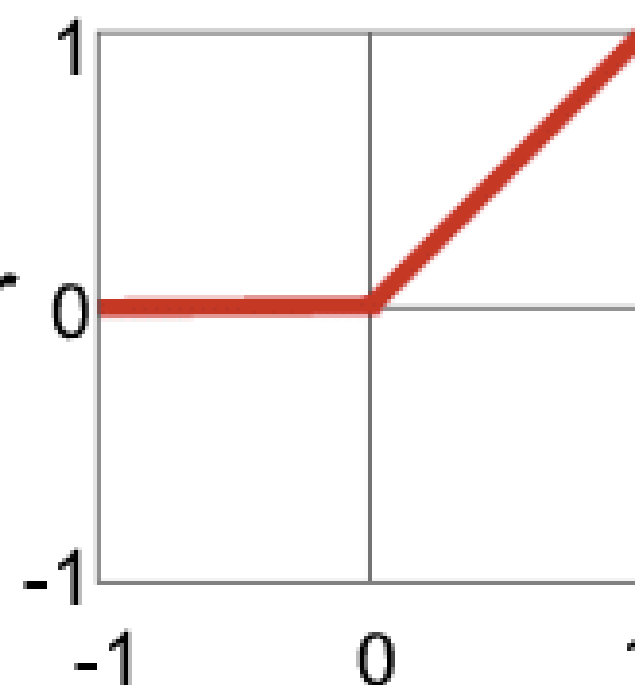
Hyperbolic Tangent



$$y = (e^x - e^{-x}) / (e^x + e^{-x})$$

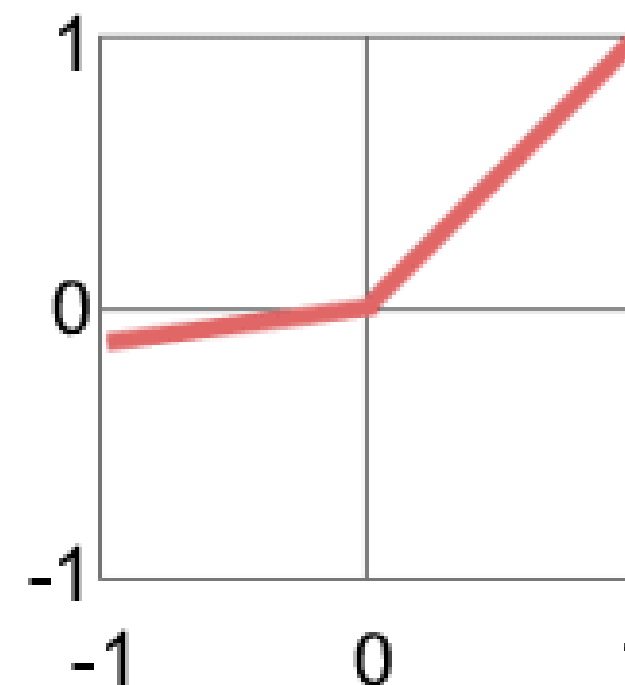
Modern Non-Linear Activation Functions

Rectified Linear Unit (ReLU)



$$y = \max(0, x)$$

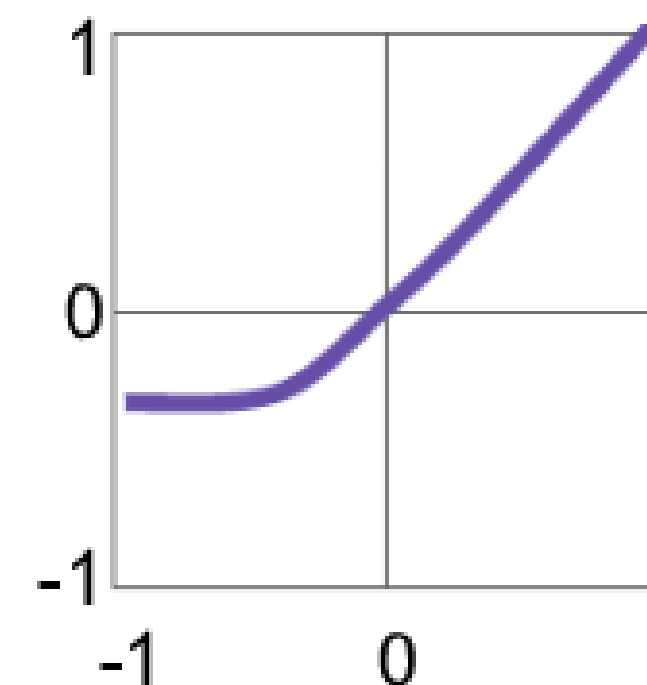
Leaky ReLU



$$y = \max(\alpha x, x)$$

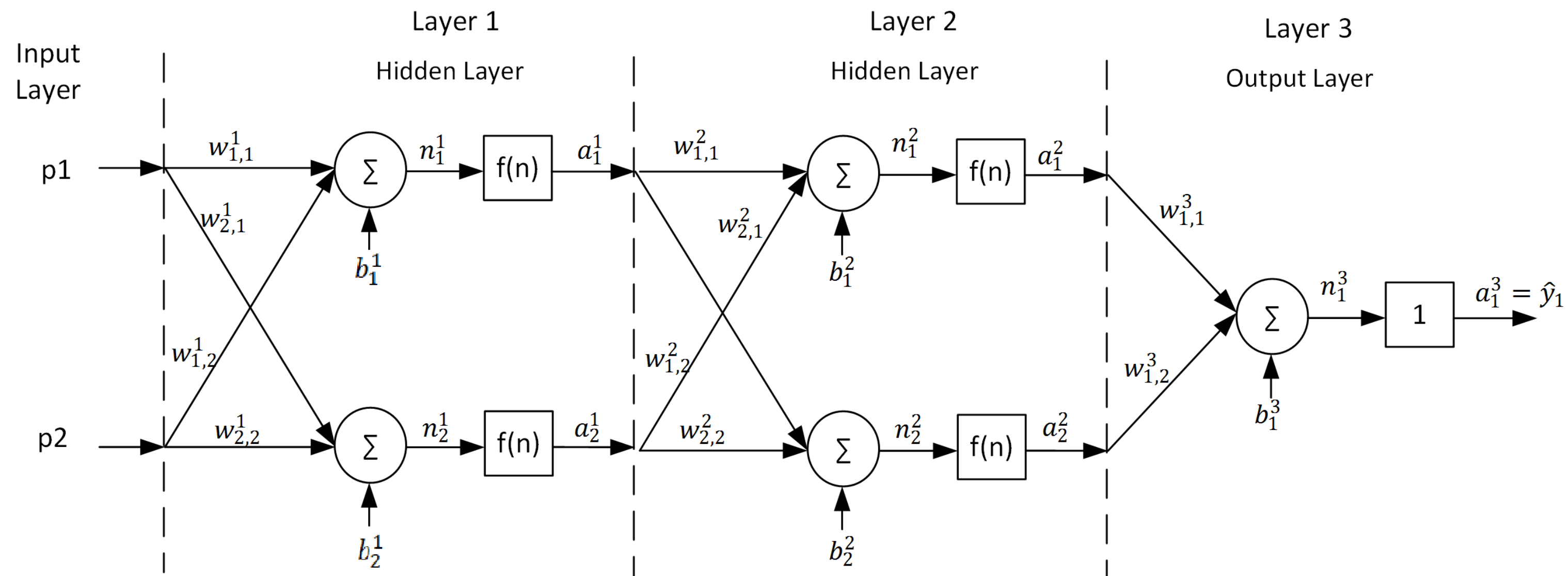
α = small const. (e.g. 0.1)

Exponential LU



$$y = \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$$

Hidden Layers

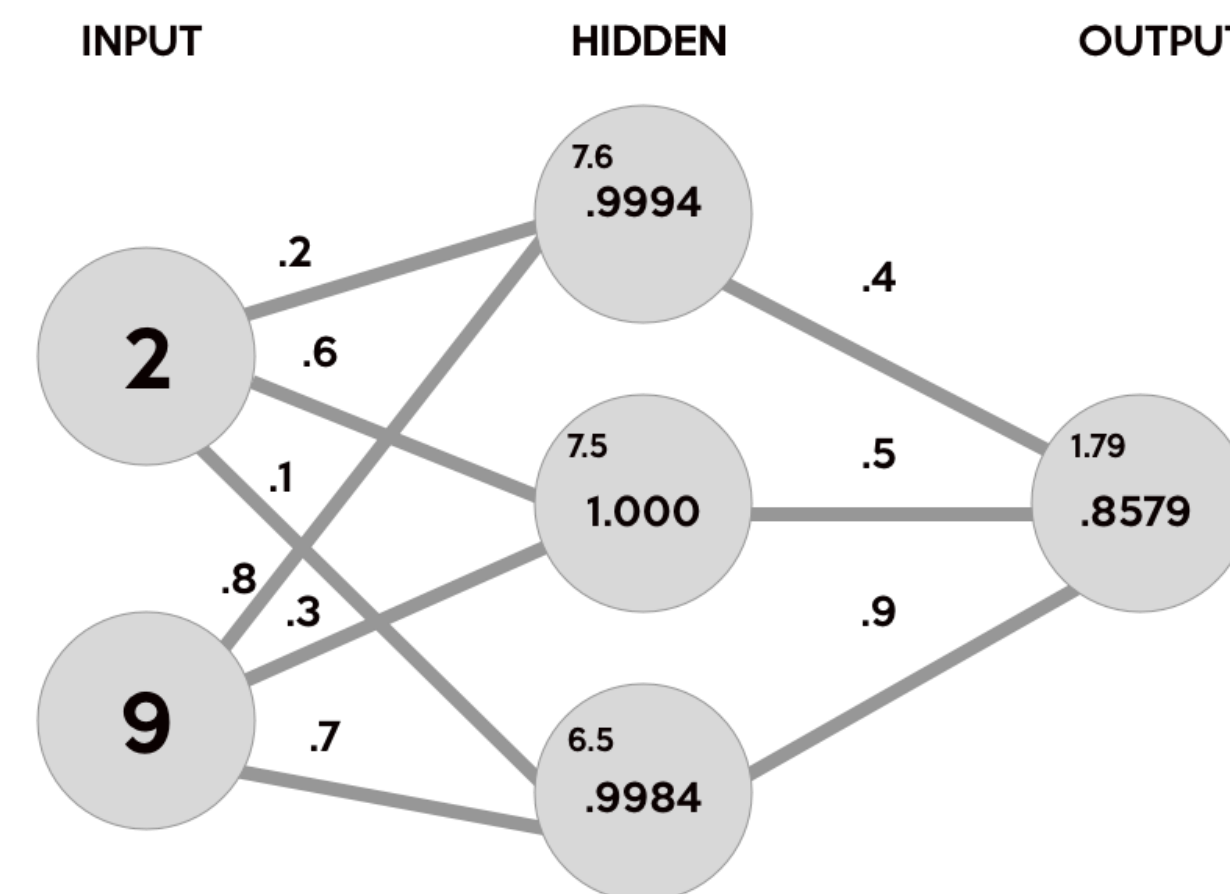


Output Layer

Si régression :

- 1 neurone en sortie
- Combinaison linéaire de la couche précédente
- Pas de fonction d'activation

Valeur obtenue = valeur prédite

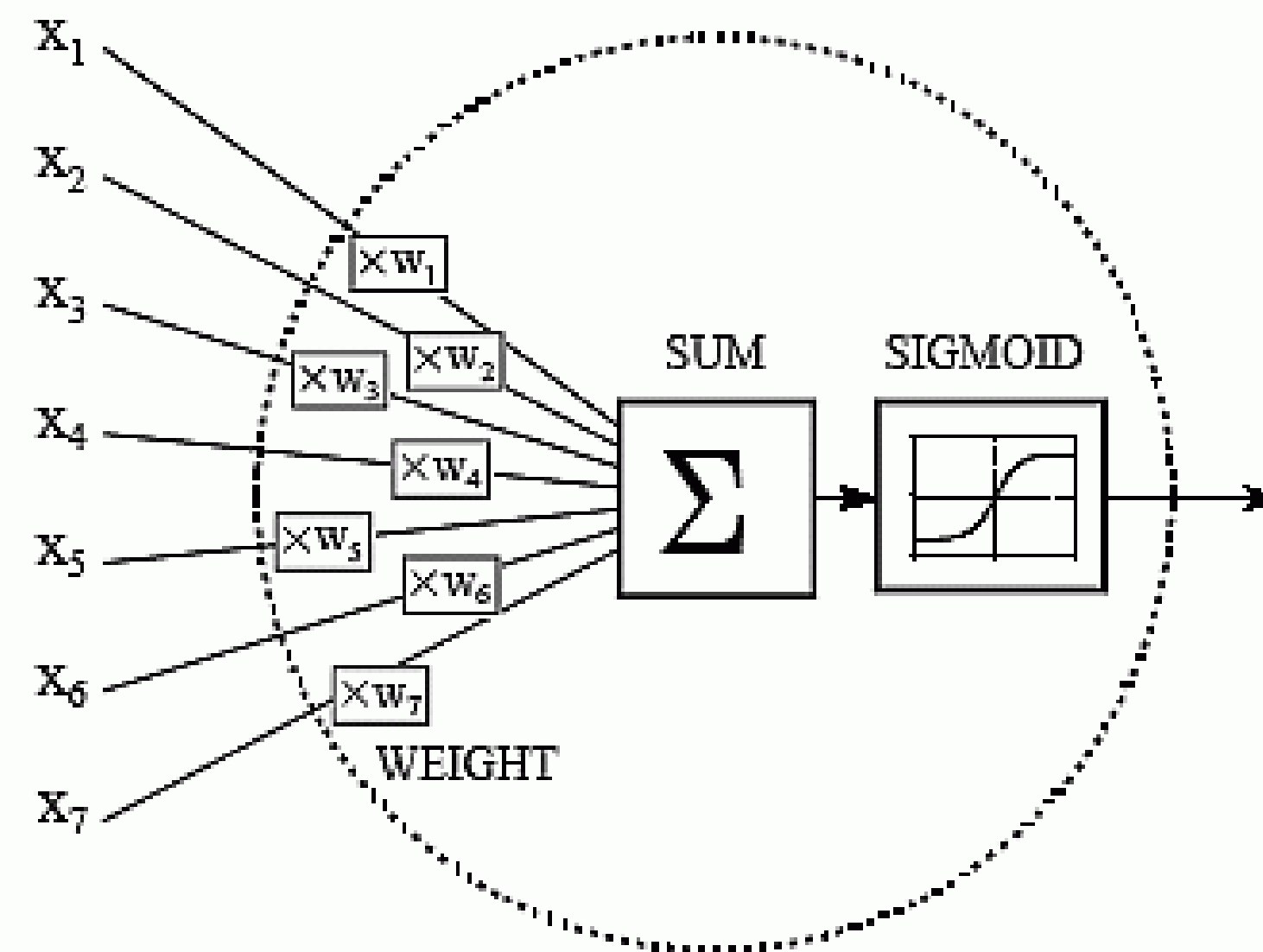


Output Layer

Si classification binaire :

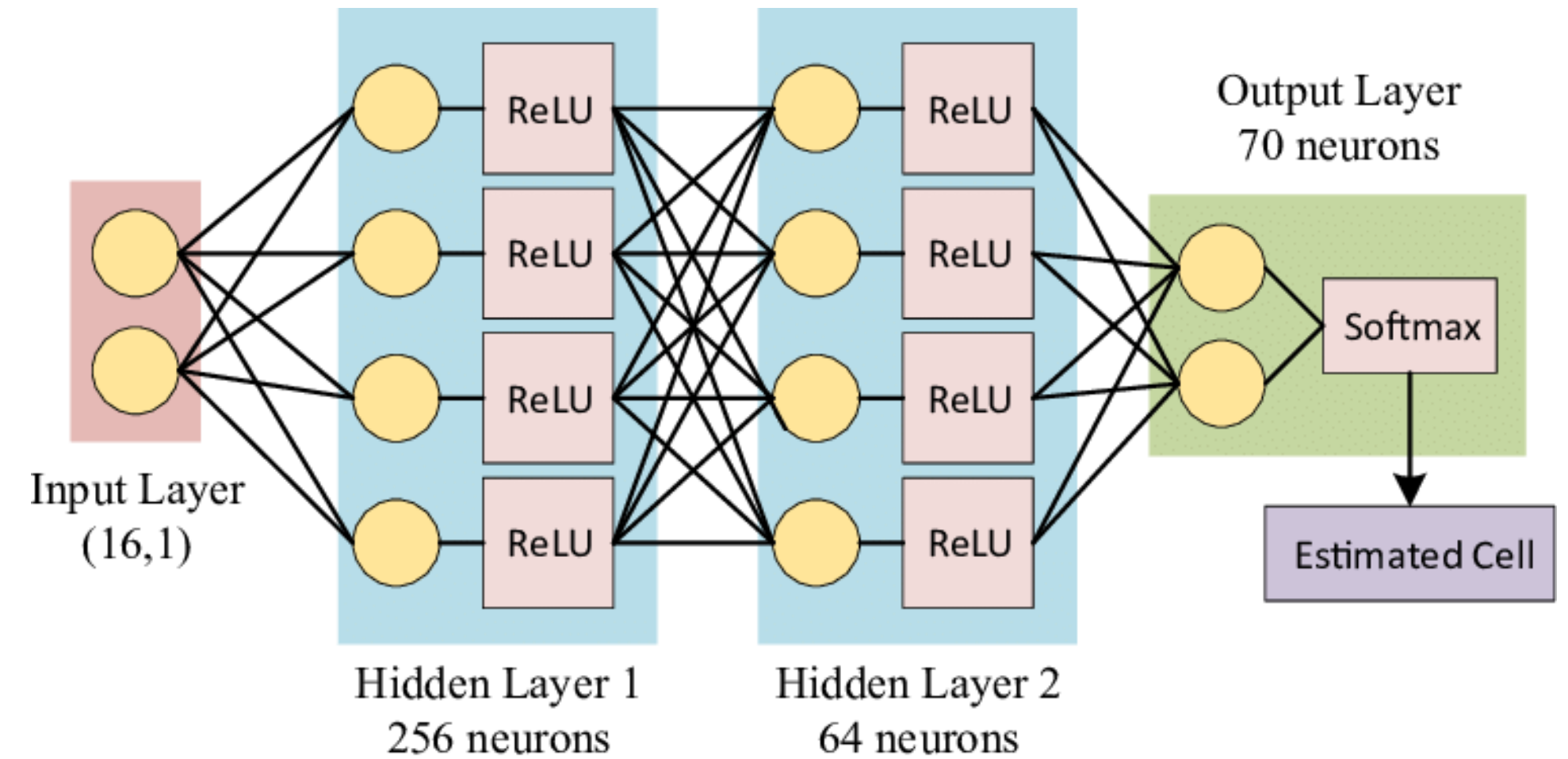
- 1 neurone en sortie
- Combinaison linéaire de la couche précédente
- Plus fonction d'activation = fonction sigmoïd

Valeur obtenue = probabilité d'appartenir à la classe positive



Définir une architecture

- Nombre de couches cachées
- Nombre de neurones par couche
- Fonction d'activation de chaque couche
- Fonction d'activation de l'output



03

Entraînement



Optimisation d'un réseau de neurones

Fonction de coût

aelion.fr

La fonction de coût (loss) mesure l'erreur entre les prédictions du réseau (output) et les vraies valeurs (cibles)

- Régression : MSE (mean_squared_error)
- Classification binaire : Log Loss (binary_crossentropy)
- Classification multi-classe : Cross Entropy (categorical_crossentropy)

Entraîner le réseau = trouver les poids qui minimisent la fonction de coût

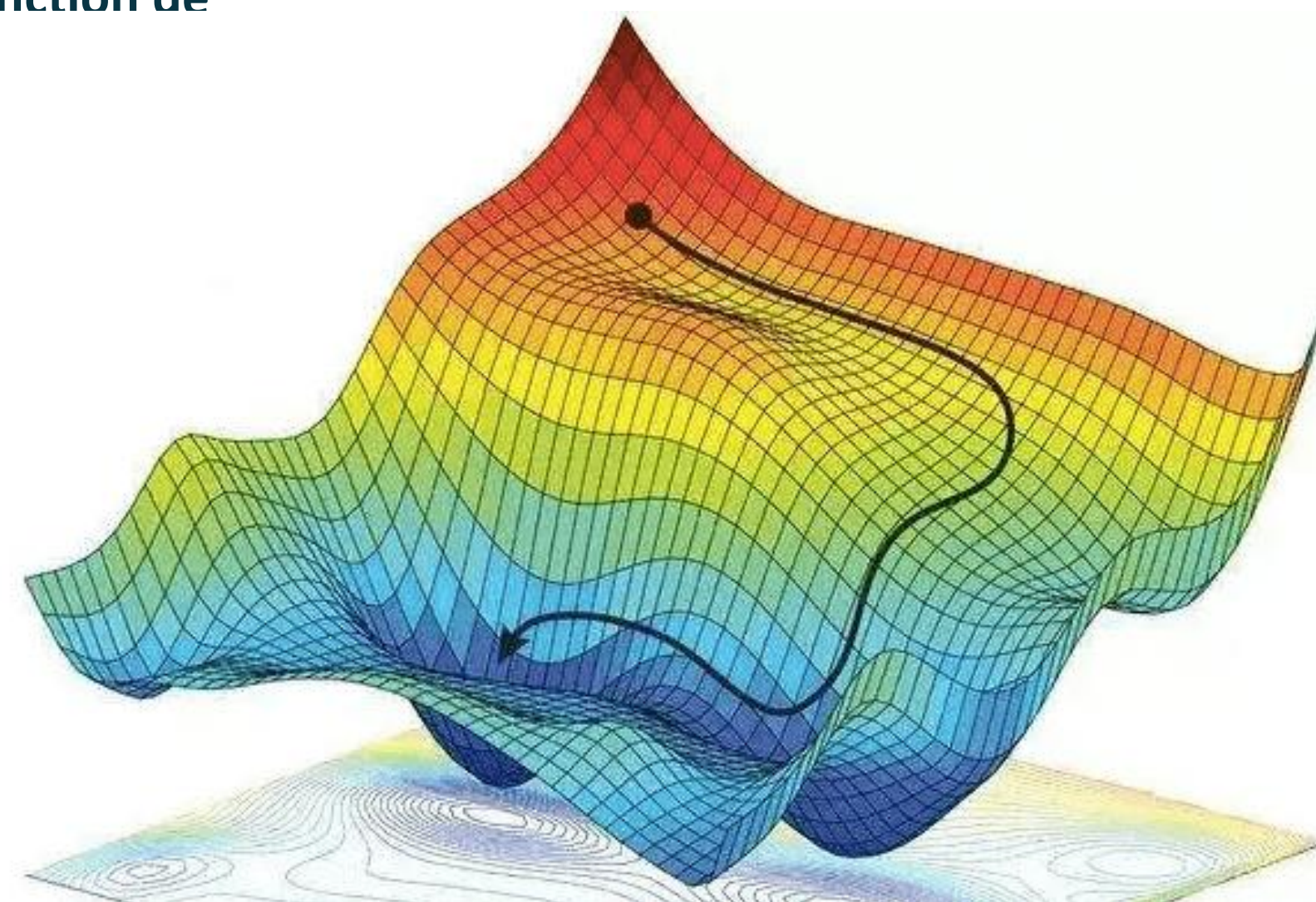
Descente de gradient

aelion.fr

Les poids sont mis à jour en fonction de :

- leur gradient (dérivée partielle)
- le taux d'apprentissage

Objectif : converger vers le minimum de la fonction de coût

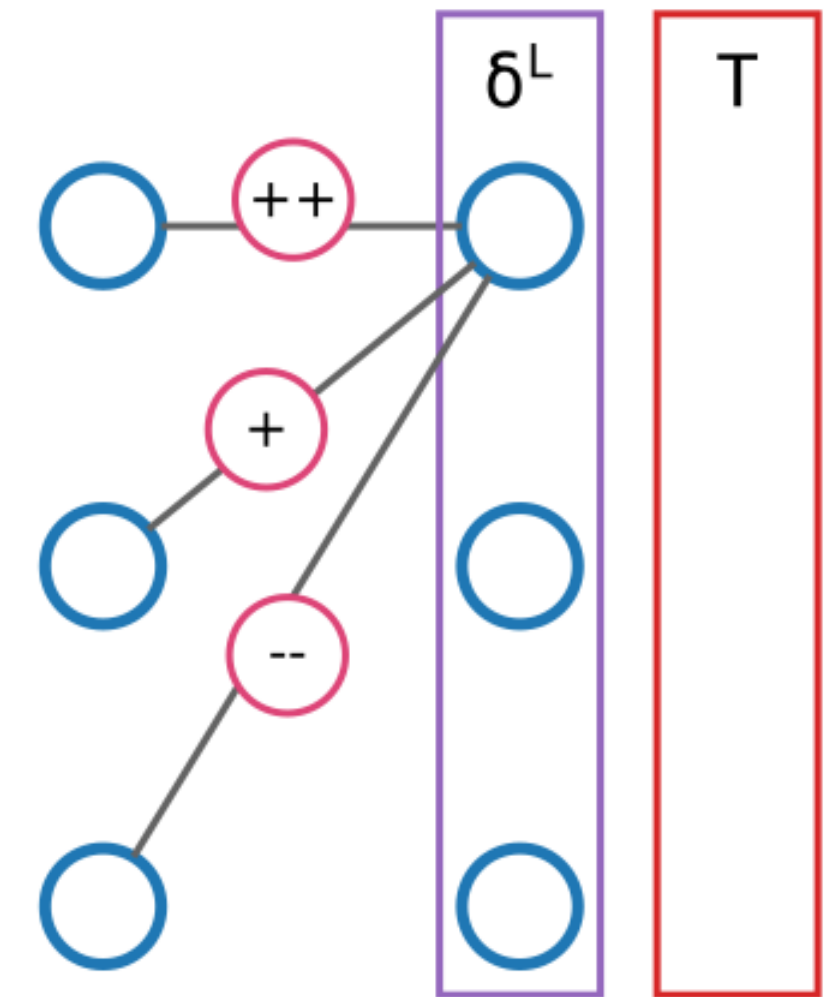
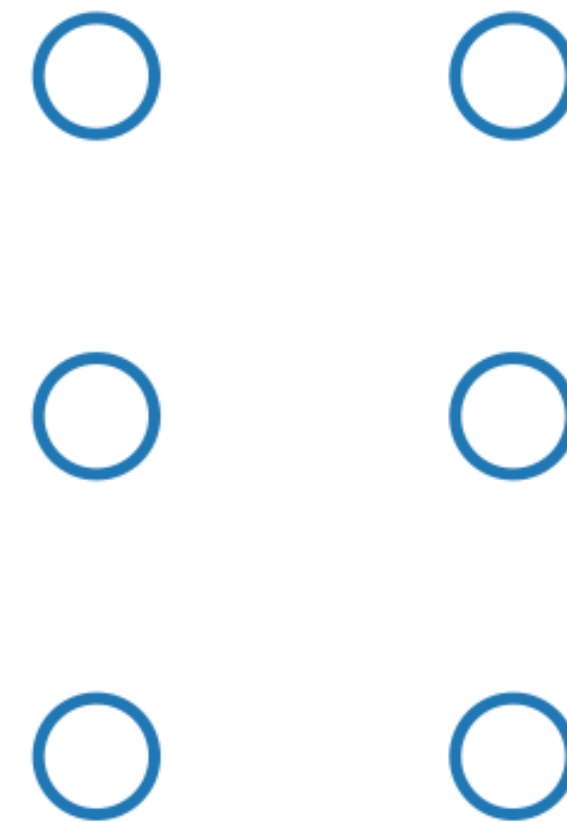


Backpropagation - première étape

aelion.fr

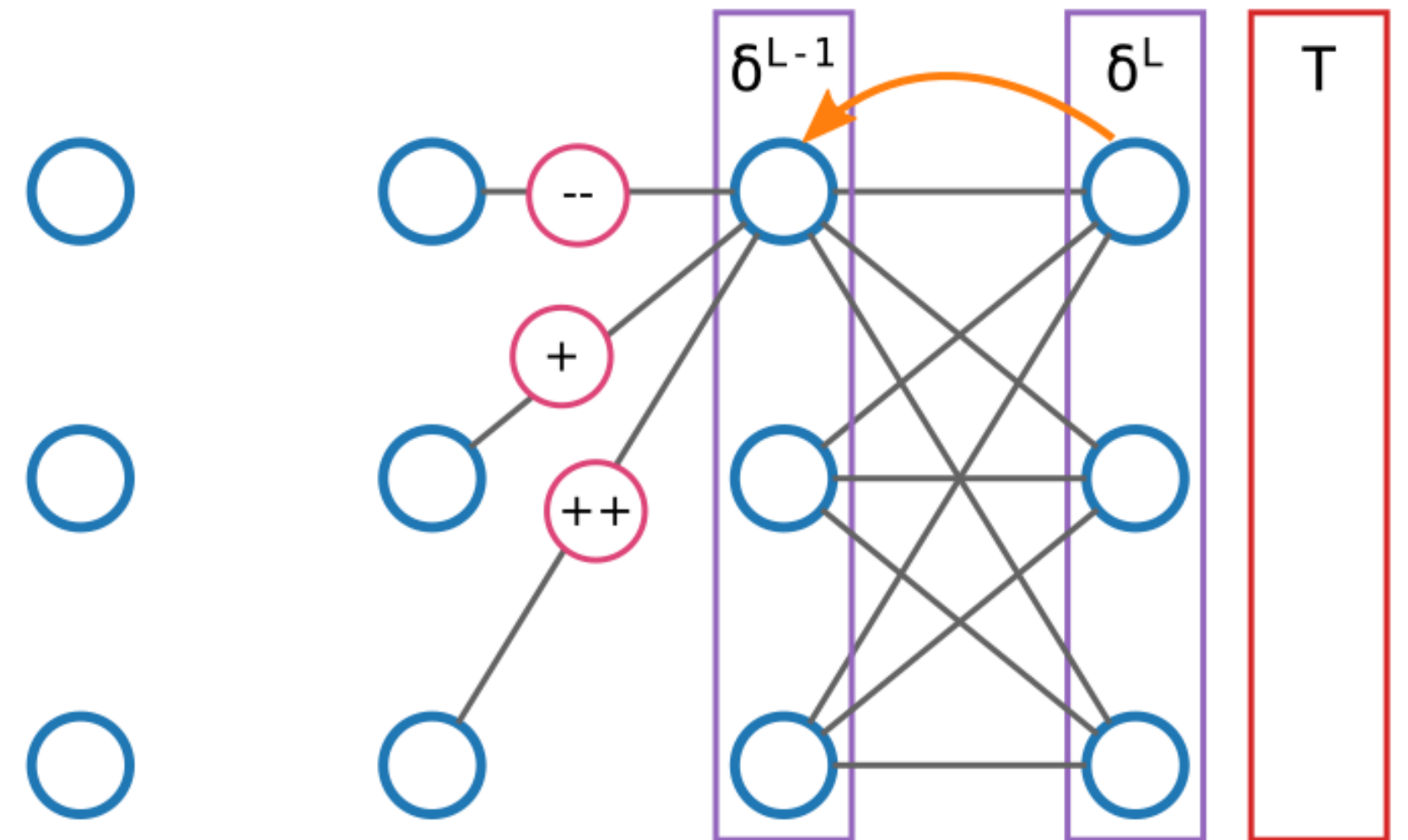
- Feedforward : propagation des données dans le réseau pour obtenir les prédictions
- Calcul de l'erreur et de la dérivée de la fonction de coût entre la prédiction et les valeurs cibles
- Calcul du gradient de chaque poids entre l'output et l'avant-dernière couche

aelion



Backpropagation - étape récurrente

- À partir des gradients calculés pour la couche n , calcul de « l'erreur » de la couche $n-1$
- Via la règle de dérivation des fonctions composées, calcul du gradient des poids de la couche $n-2$ vers $n-1$
- **Mise à jour des poids**



Descente de gradient stochastique

aelion.fr

Le gradient est calculé à partir du coût global :

- à chaque observation correspond une erreur (carré des différences, log de la probabilité...)
- coût global = somme des erreurs sur toutes les observations

Mais calculer le coût global sur des centaines de millions de données est très coûteux !

Solution : calcul du gradient et mise à jour des poids à partir de :

- une seule donnée — SGD (Stochastic Gradient Descent)
- un lot de données — mini-batch gradient descent

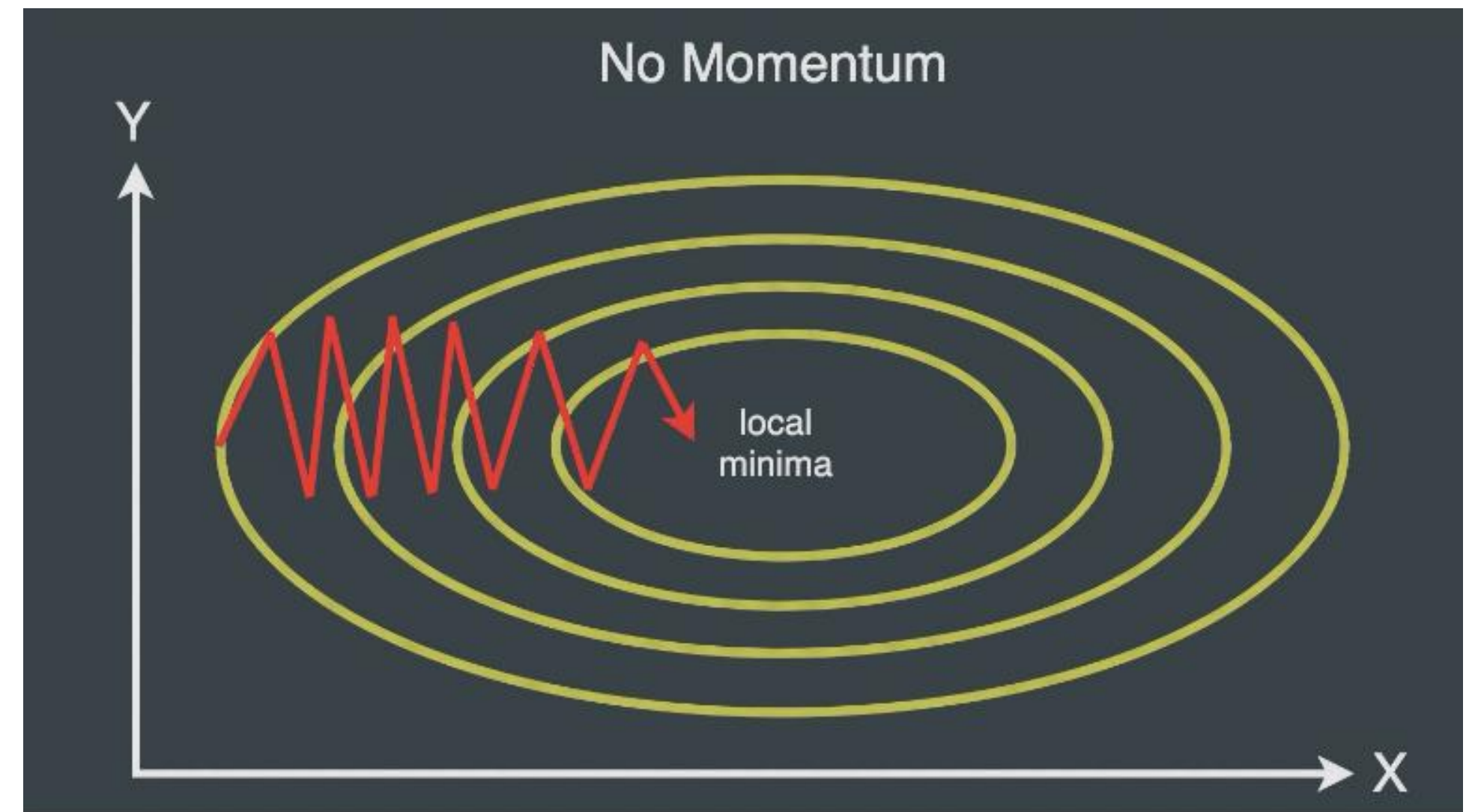
Momentum

aelion.fr

- Ajout d'une fraction de la mise à jour précédente (« vitesse »)
- Évite les oscillations chronophages et permet de converger plus vite

Visualisation du momentum en action disponible en ligne

aelion



RMSProp

aelion.fr

- Les gradients peuvent avoir des échelles très différentes
- Idée : un taux d'apprentissage adaptatif pour chaque poids
- Pour cela : taux d'apprentissage unique mais gradient « standardisé »
 - « standardiser » = diviser le gradient par la racine carrée de la moyenne mobile du carré des gradients précédents

Root Mean Squared Propagation = RMSProp

- Accélère la convergence et évite les « plateaux »

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) \left(\frac{\delta C}{\delta w} \right)^2$$

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{E[g^2]_t}} \frac{\delta C}{\delta w}$$

Adam

Momentum + RMSProp = Adam

- Fonctionne très bien dans une large majorité de cas !

Commencer avec Adam et les paramètres suivants :

- Momentum (β_1) = 0,9
- RMSProp (β_2) = 0,999
- Taux d'apprentissage entre 0,001 et 0,0001

**STOCHASTIC
GRADIENT
DESCENT**



**SGD WITH
MOMENTUM**



RMSPROP



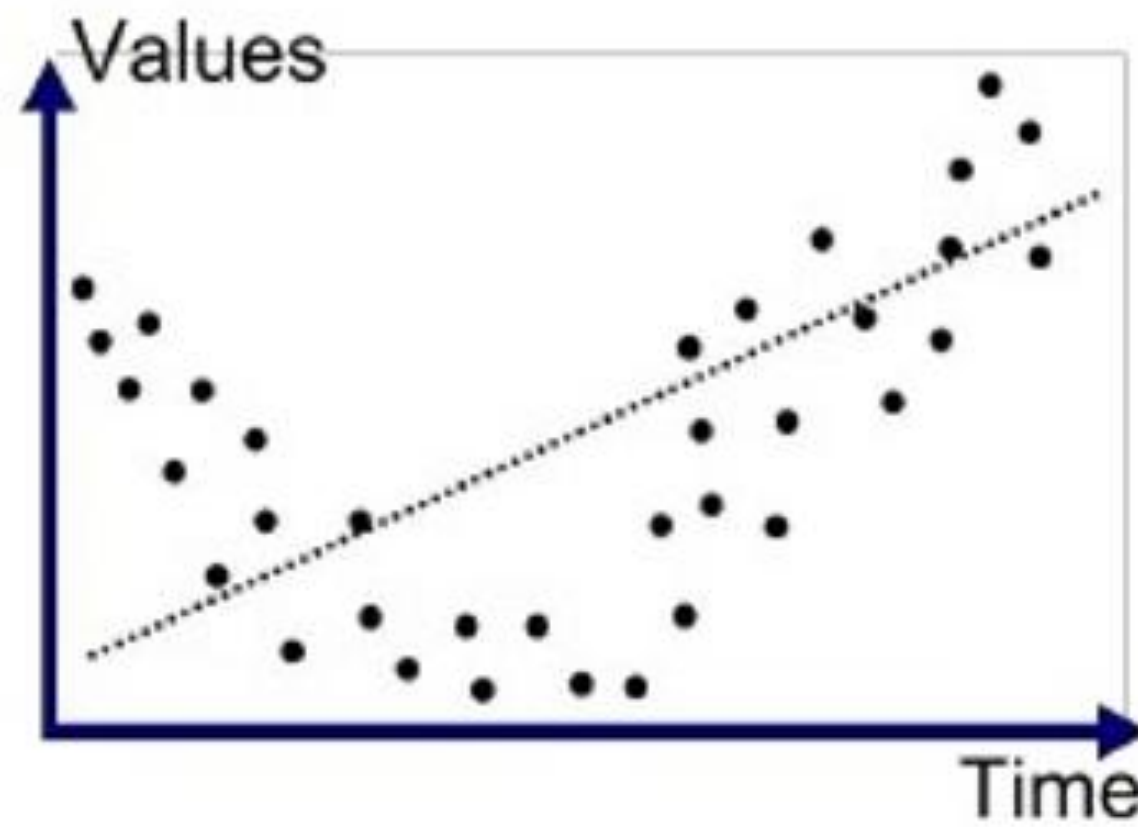
ADAM



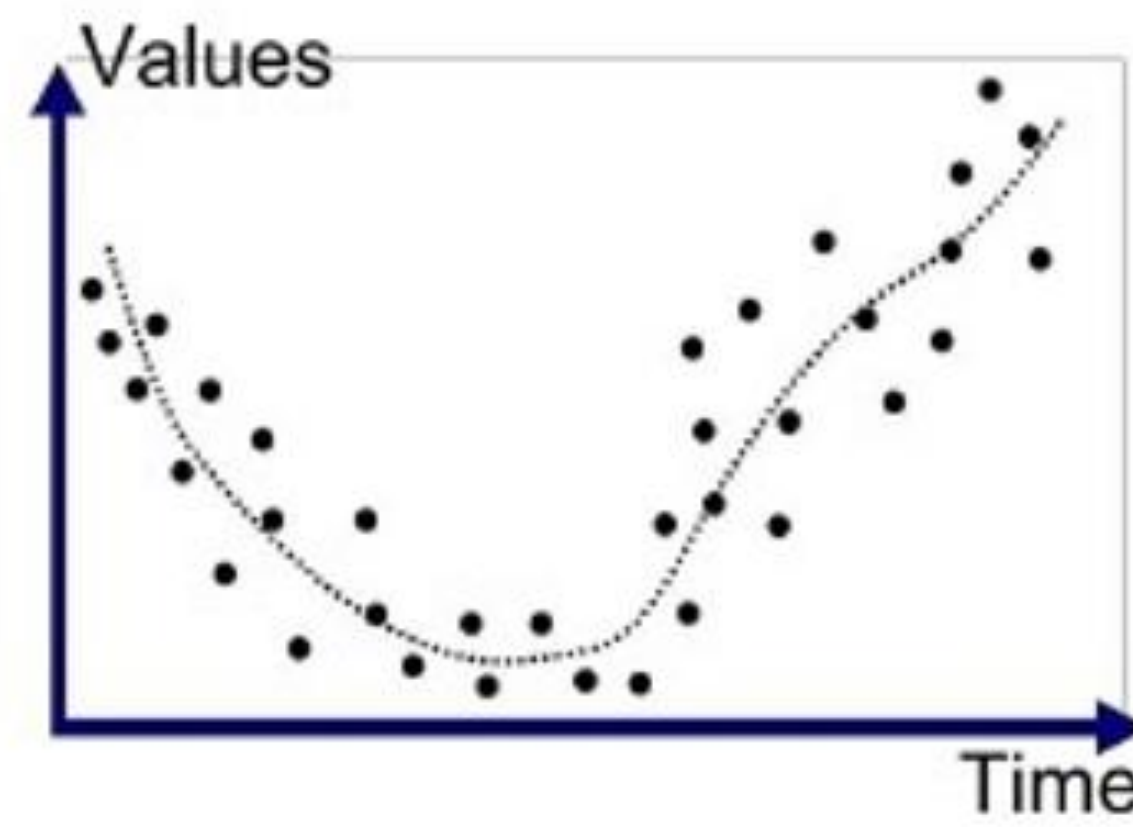
Surapprentissage (overfitting)

aelion.fr

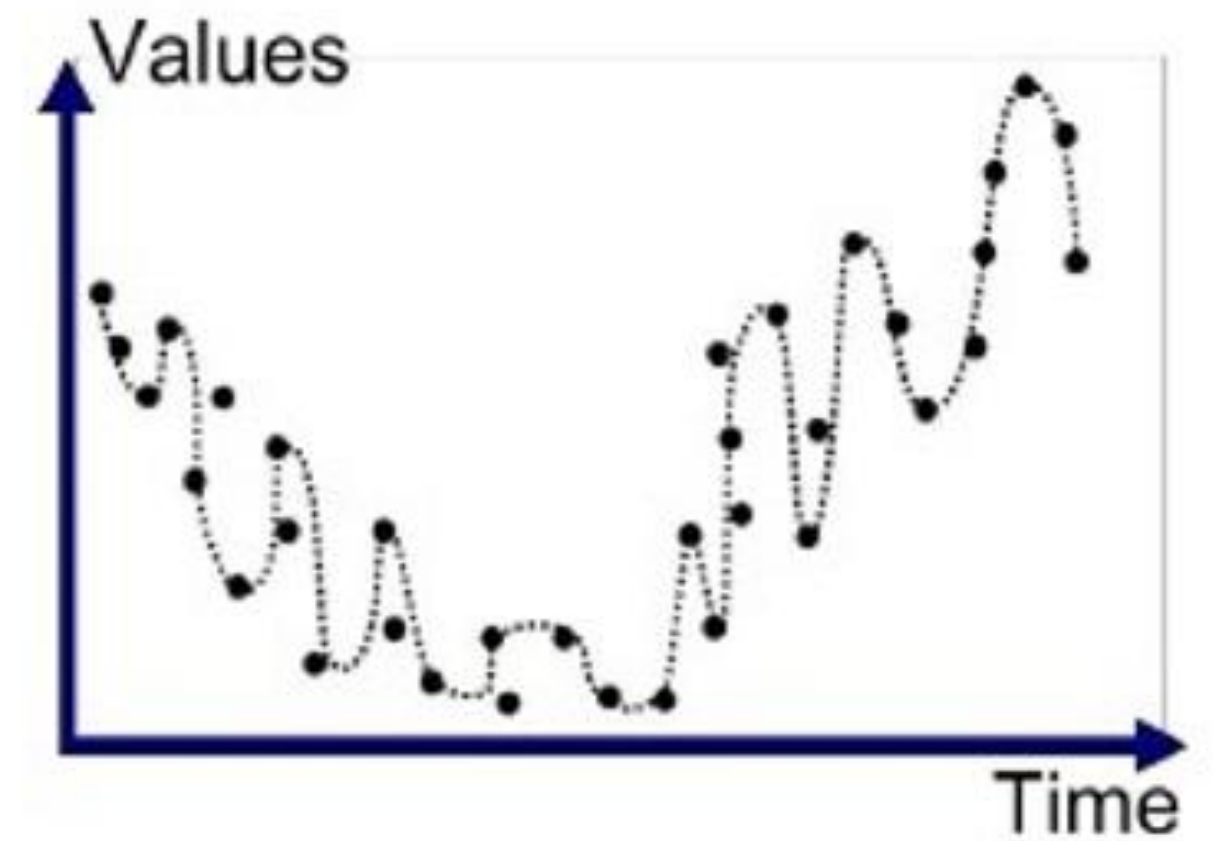
aelion



Underfitted



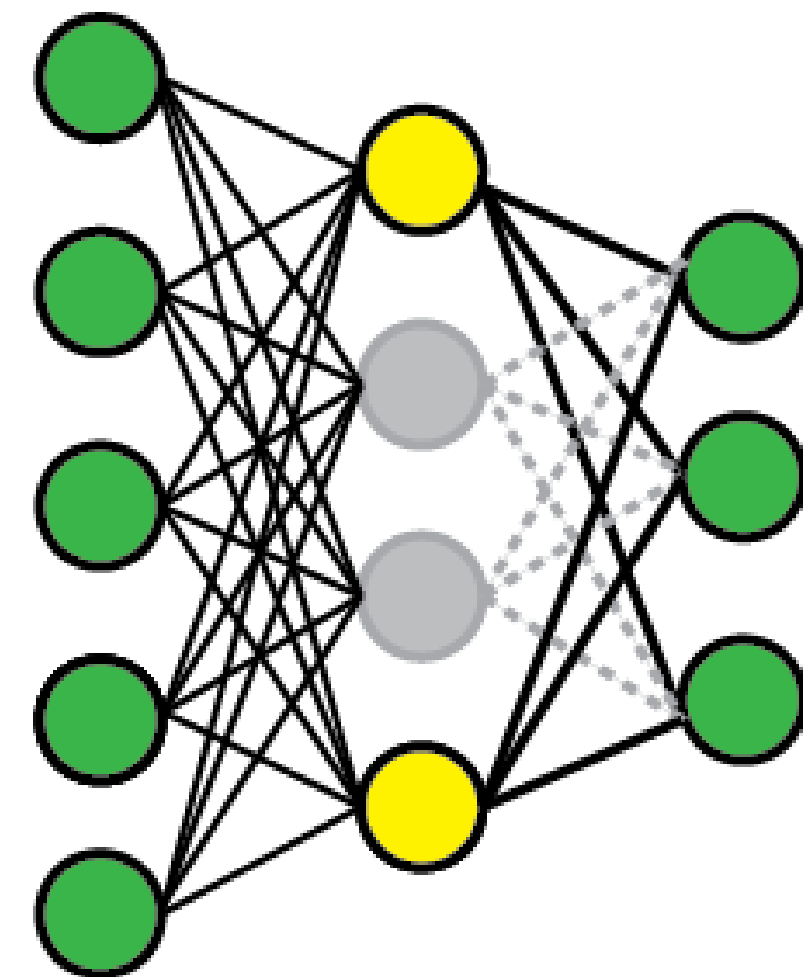
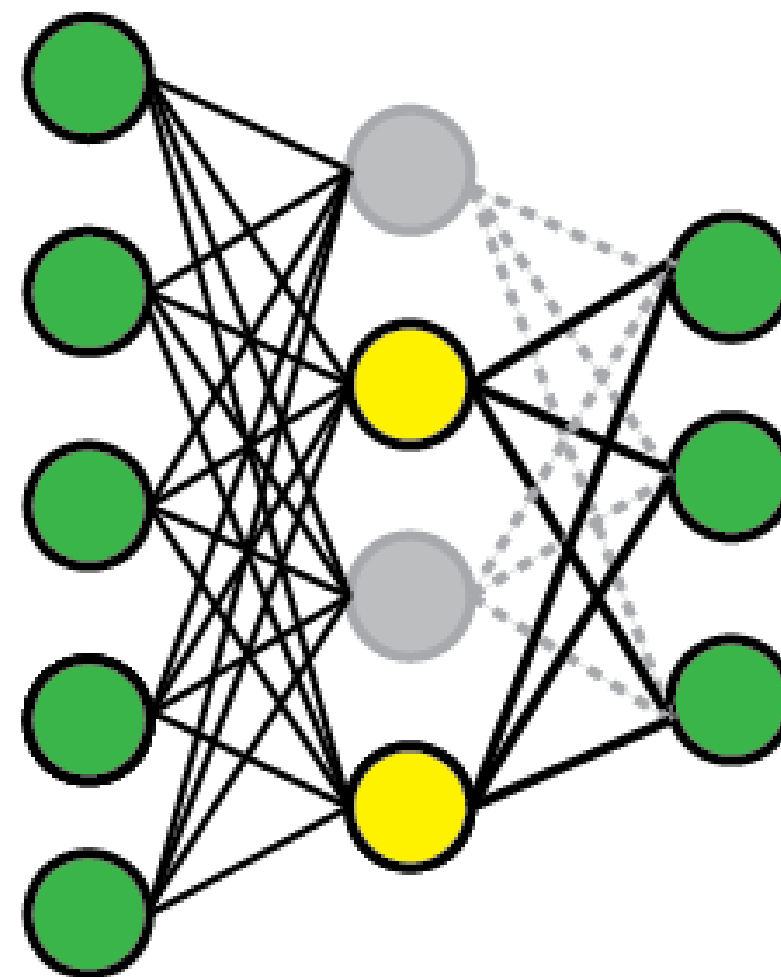
Good Fit/Robust



Overfitted

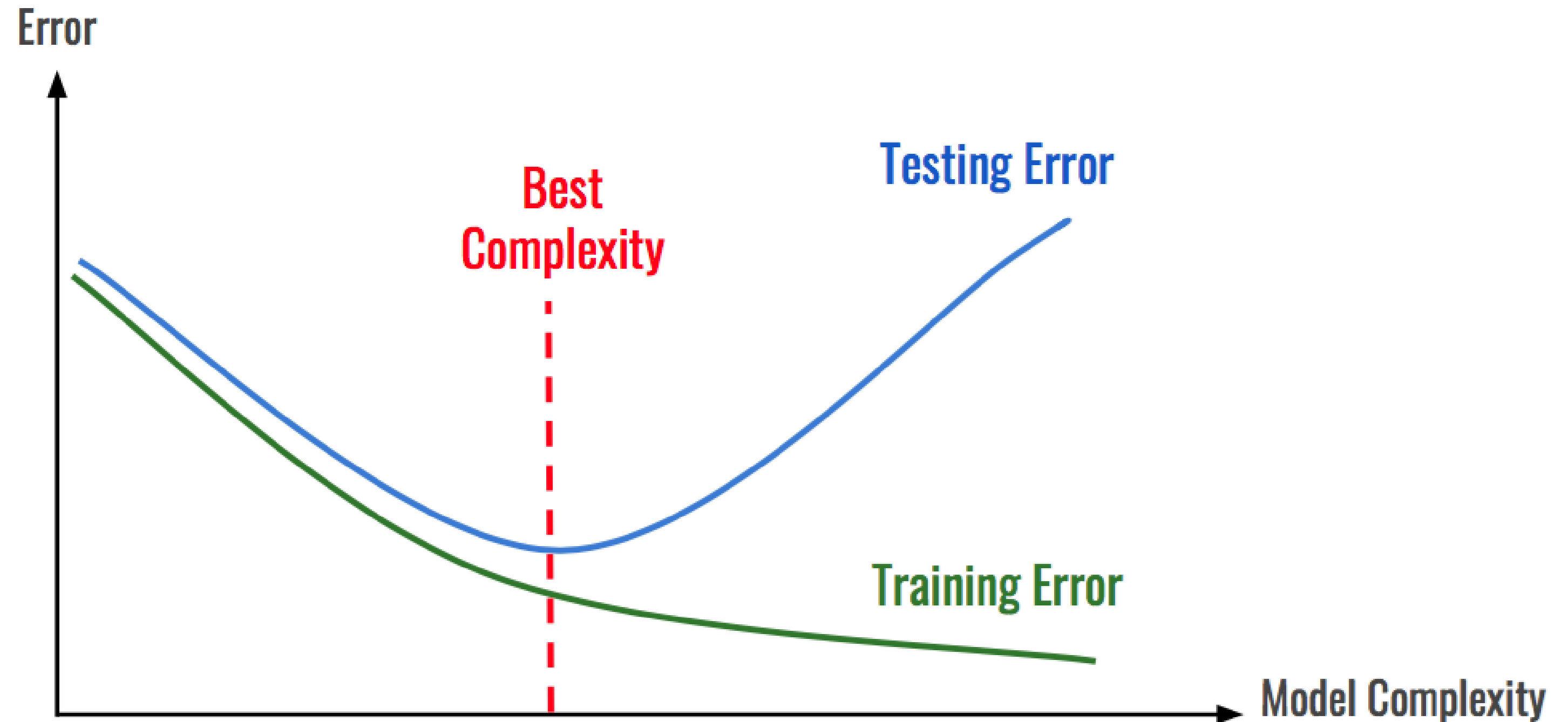
Dropout

- À chaque passe, certains neurones sont ignorés
- Force la couche suivante à ne pas dépendre de quelques neurones : tous les neurones doivent être utiles
- **Meilleure généralisation = moins de surapprentissage**



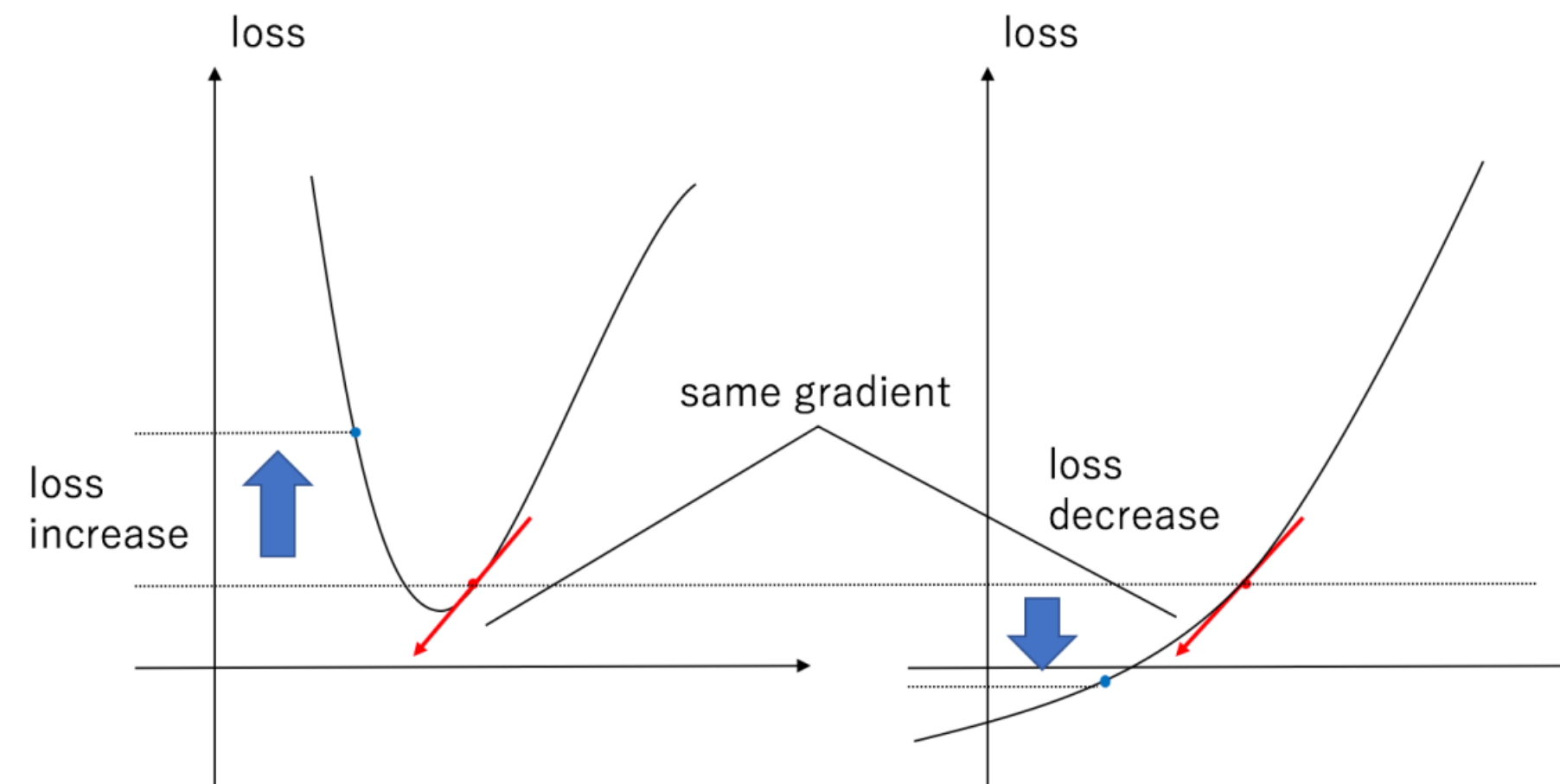
Early Stopping

- À chaque mini-batch, on évalue le critère d'évaluation sur le jeu de validation
- Quand celui-ci ne s'améliore plus après un certain nombre de mini-batch, on arrête l'entraînement
- **On revient aux poids de la meilleure itération**



Normalisation et standardisation

- Si les features ont des échelles différentes, la convergence risque d'être très lente !
- **C'est valable aussi entre les couches cachées...**



Batch normalization

Batch normalization = normaliser et standardiser la sortie d'une couche

- Idée : remettre toutes les valeurs de sortie à la même échelle
- Stabilise les sorties de la couche, donc les entrées de la suivante :
 - pas de grosses fluctuations sur les poids de la couche suivante
 - accélère l'entraînement et permet un taux d'apprentissage plus élevé
 - rend la couche suivante plus « indépendante » de la précédente
 - rend le modèle plus généralisable et évite l'overfitting

Comment le mettre en place ?

aelion.fr

- À insérer entre la combinaison linéaire et la fonction d'activation
- Pas besoin de constante dans la régression linéaire
- **Dans Keras, c'est une nouvelle couche :**

```
model.add(layers.Dense(64, use_bias=False))  
model.add(layers.BatchNormalization())  
model.add(Activation("relu"))
```

aelien

04

CNN



Convolutional Neural Network

Convolution

aelion.fr

- Image = matrice de pixels
- Varie énormément selon l'angle de prise de vue, l'éclairage, l'arrière-plan...
- Nécessite une extraction de features pour être exploitée
- **Filtre (masque de convolution) = détecteur de caractéristique = petite matrice : 3×3 , 5×5 ...**
- Convolution = multiplication élément par élément du filtre et d'une fenêtre glissante sur l'image

Convolution

Input

4	9	2	5	8	3
5	6	2	4	0	3
2	4	5	4	5	2
5	6	5	4	7	8
5	7	7	9	2	1
5	8	5	3	8	4

$$n_H \times n_W = 6 \times 6$$

Filter

1	0	-1
1	0	-1
1	0	-1

*

=

Result

2			

2 = $4 \times 1 + 9 \times 0 + 2 \times (-1) +$
 $5 \times 1 + 6 \times 0 + 2 \times (-1) +$
 $2 \times 1 + 4 \times 0 + 5 \times (-1)$

Convolution

Input

4	9	2	5	8	3
6	2	4	0	3	
2	4	5	4	5	2
5	6	5	4	7	8
5	7	7	9	2	1
5	8	5	3	8	4

$$n_H \times n_W = 6 \times 6$$

Filter

1	0	-1
1	0	-1
1	0	-1

*

=

Result

2	6		

6

$$= 9 \cdot 1 + 2 \cdot 0 + 5 \cdot (-1) + 6 \cdot 1 + 2 \cdot 0 + 4 \cdot (-1) + 4 \cdot 1 + 5 \cdot 0 + 4 \cdot (-1)$$

Masque de convolution

aelion.fr

aelion

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

*

1	1	1
0	0	0
-1	-1	-1

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

Exemple de détecteur de traits horizontaux

Exemple de features détectées

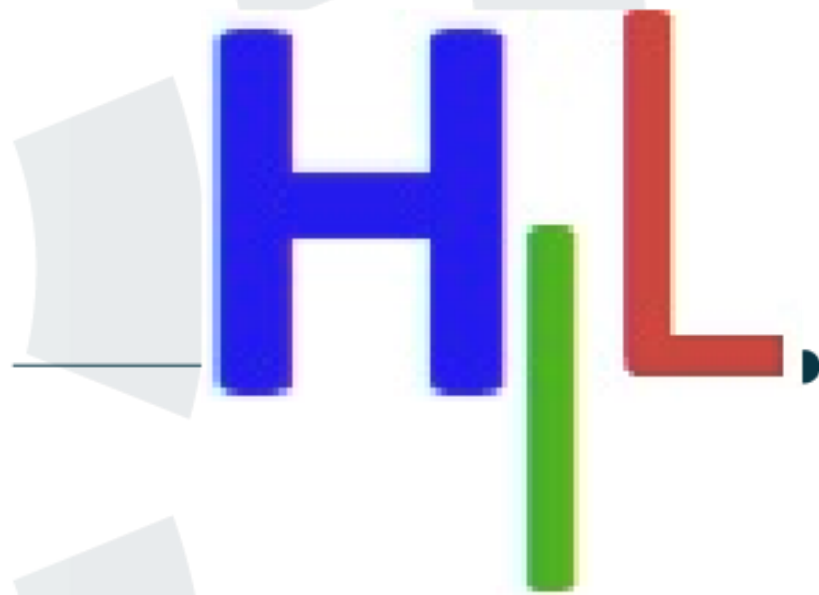


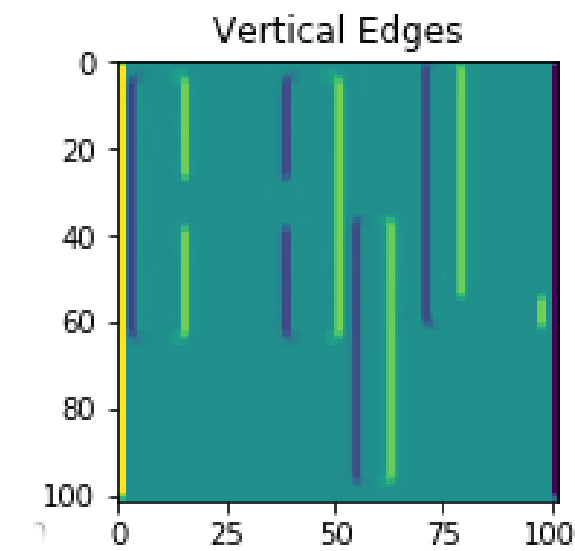
Image de départ

1	0	-1
1	0	-1
1	0	-1

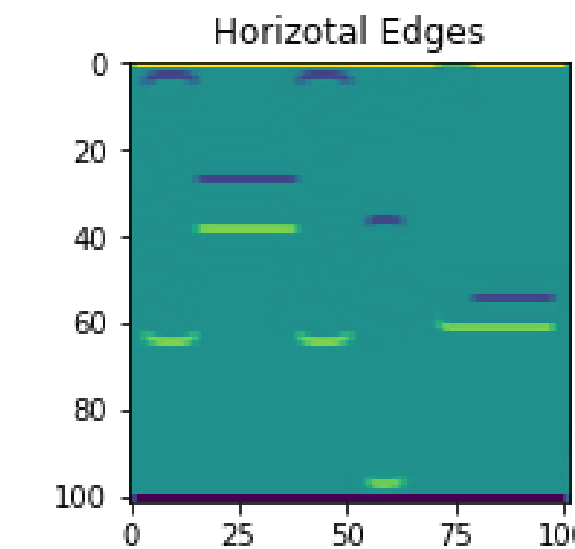
Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

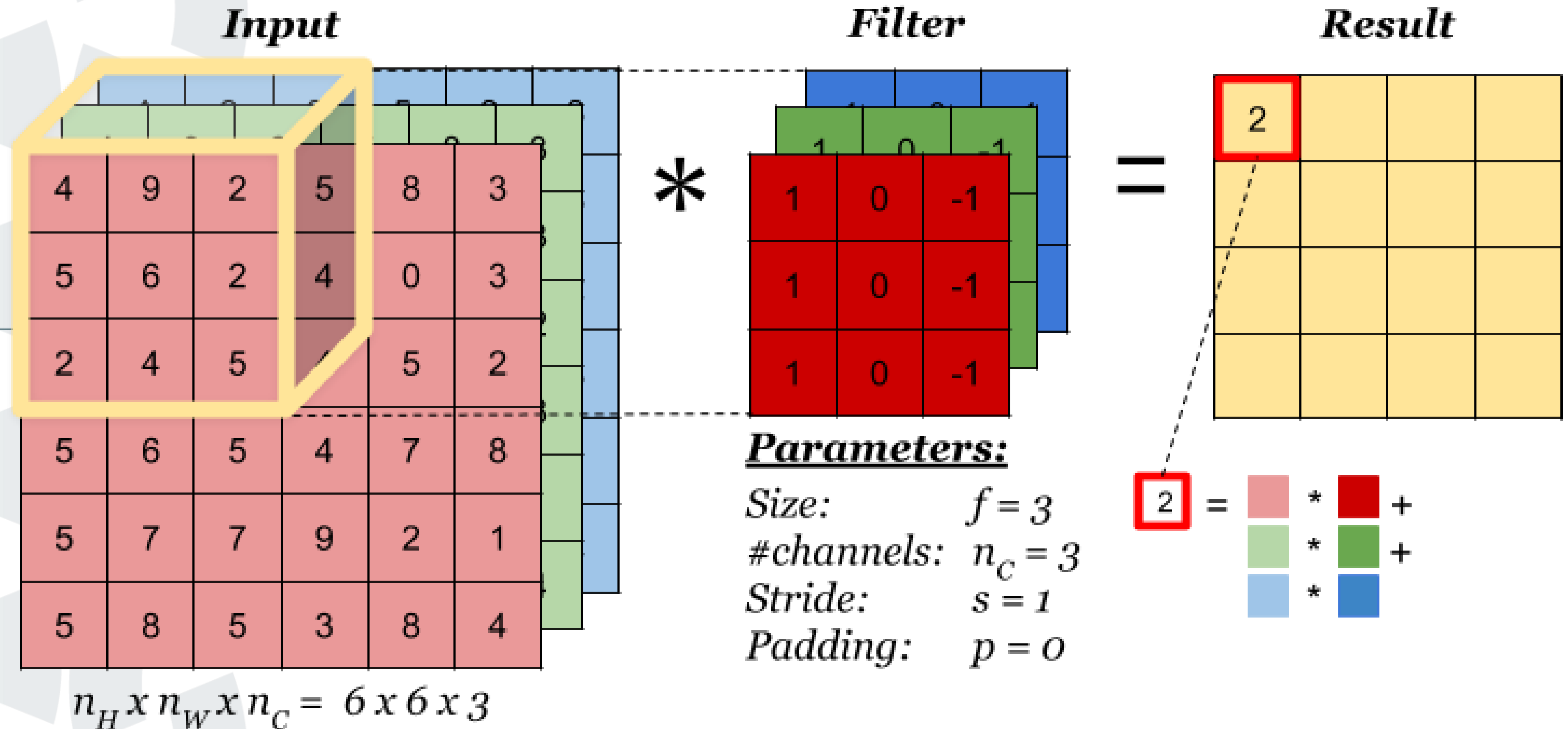


Détection de traits verticaux



Détection de traits horizontaux

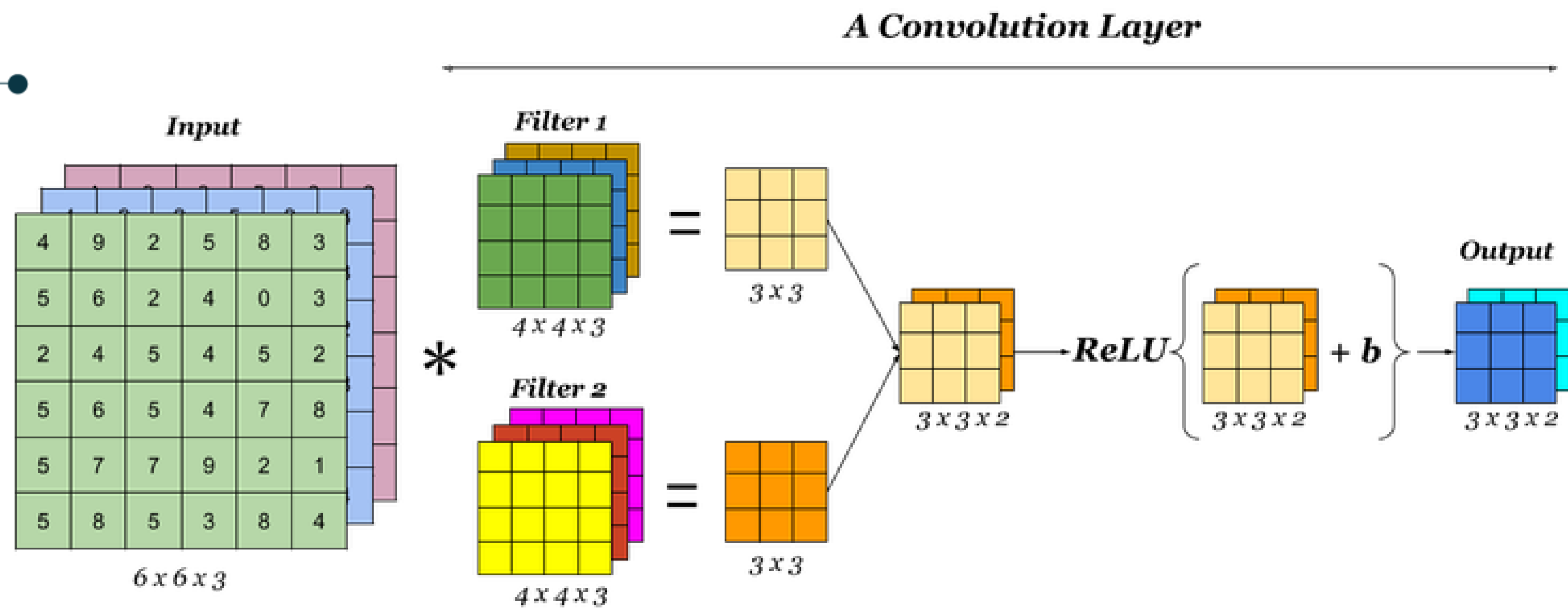
Convolution couleurs



Sur une image RVB, le filtre possède une profondeur égale au nombre de canaux

Couche complète de convolution

- Plusieurs filtres sont utilisés
- Suivis d'une fonction d'activation



Couche d'aplatissement

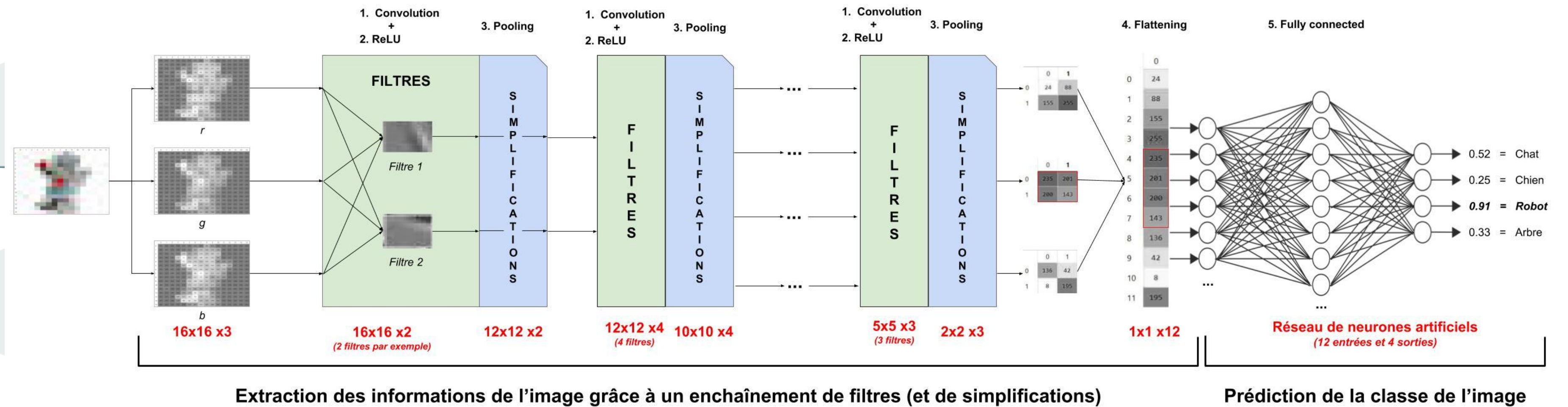
- En fin de CNN : plusieurs matrices
- Transformation en un seul vecteur (couche Flatten dans Keras)
- Ce vecteur = couche d'entrée d'un réseau « classique » avec des couches fully-connected (couches Dense)



Exemple de réseau complet

aelion.fr

aelion



Extraction des features (convolutions + pooling) puis prédiction de la classe

05

CNN — avancé

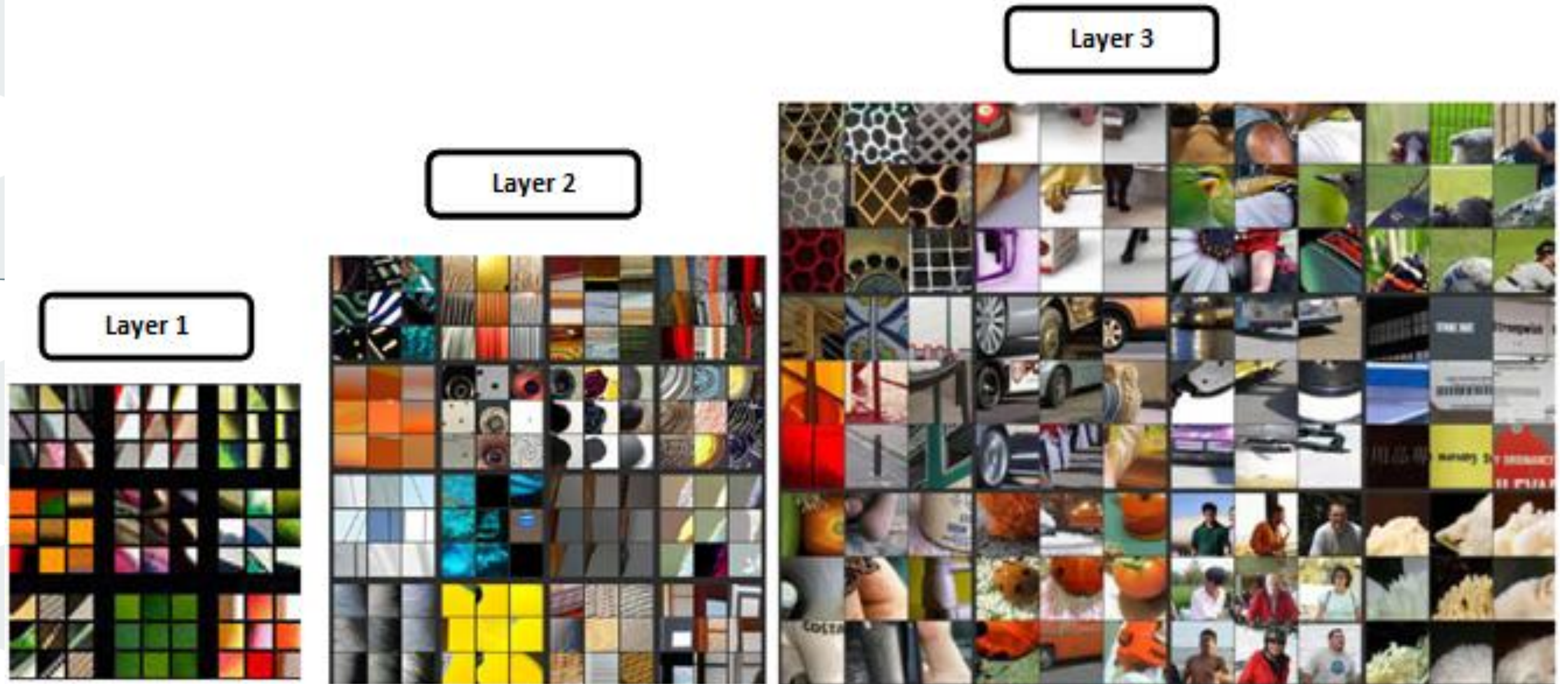


Considérations et architectures profondes

Niveaux d'abstraction — visualisation

aelion.fr

aelion

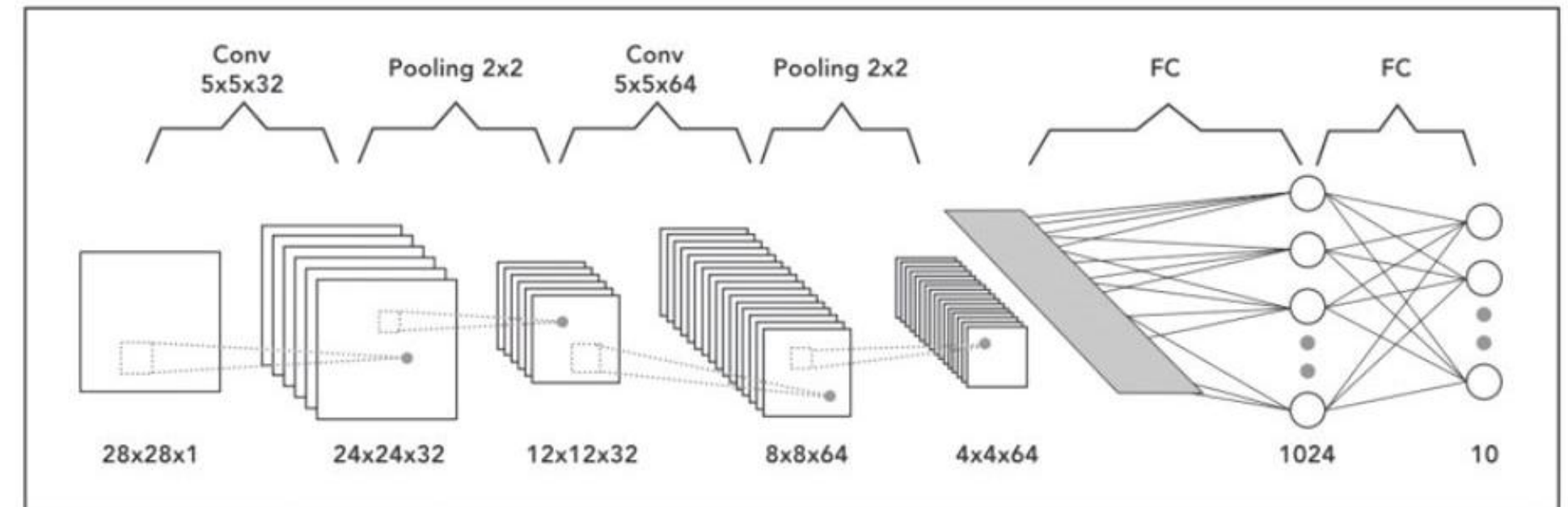


Principes d'architecture

aelion.fr

- Plus de features complexes que de features basiques
- **Le nombre de filtres augmente avec la profondeur !**
- La taille de l'image est réduite petit à petit pour converger vers une caractérisation unique

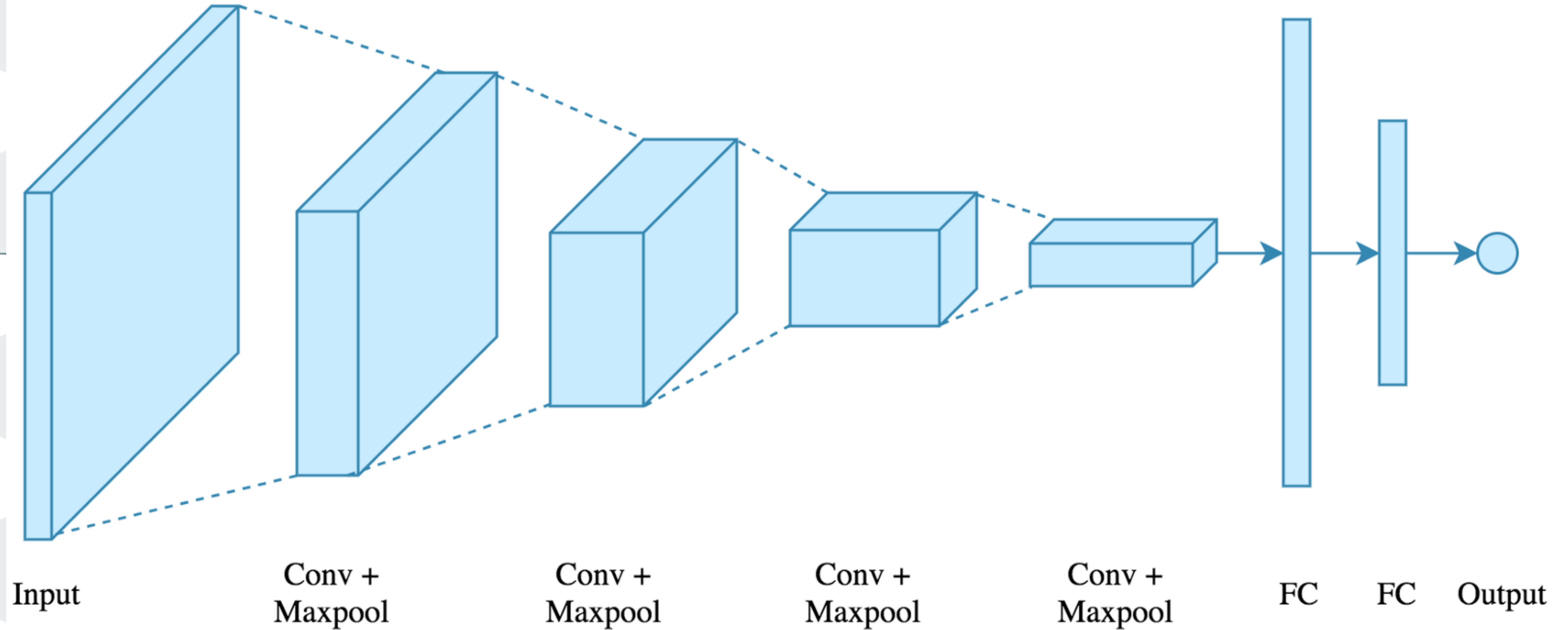
aelion



Architectures classiques

aelion.fr

aelion

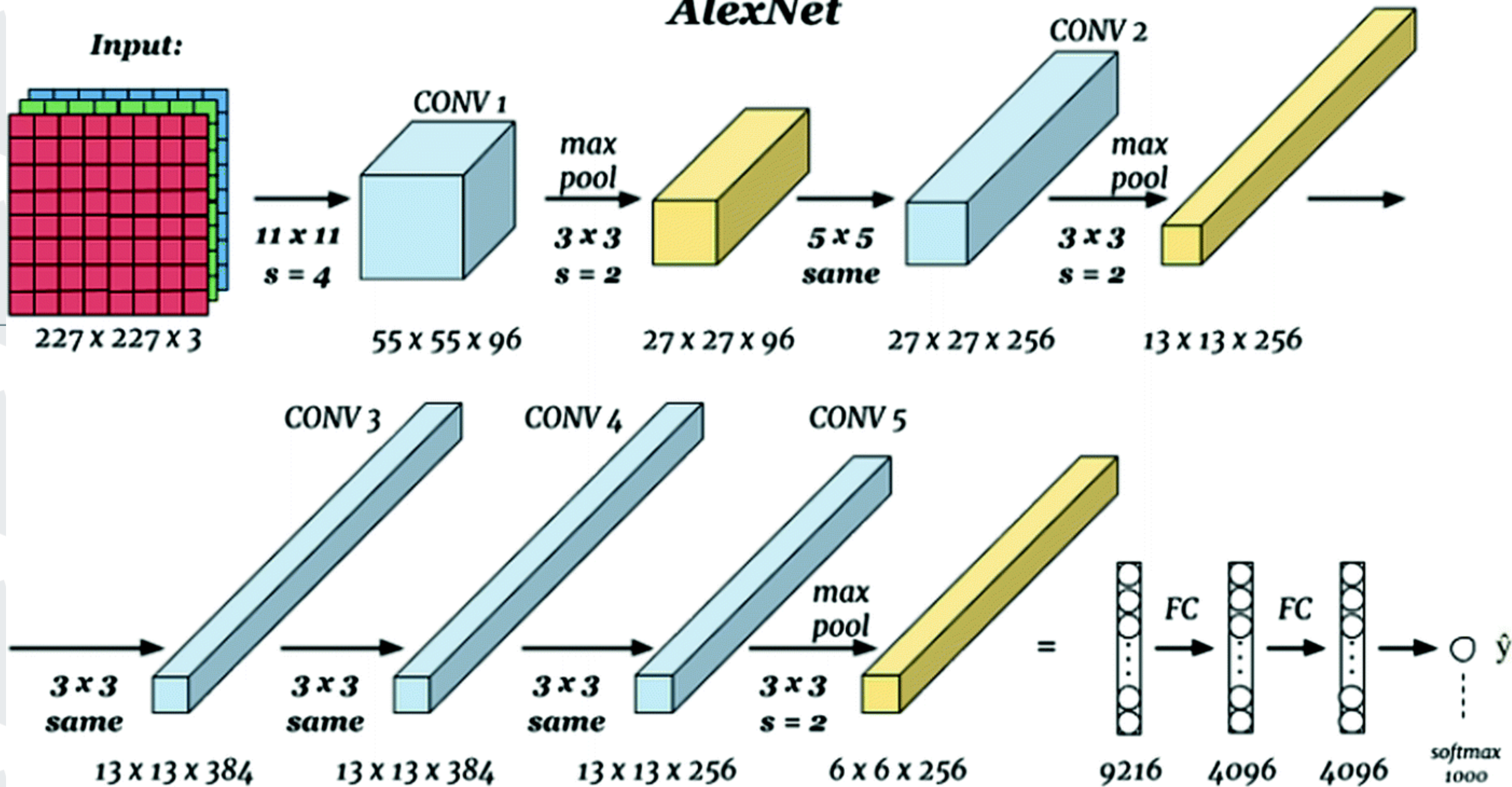


AlexNet

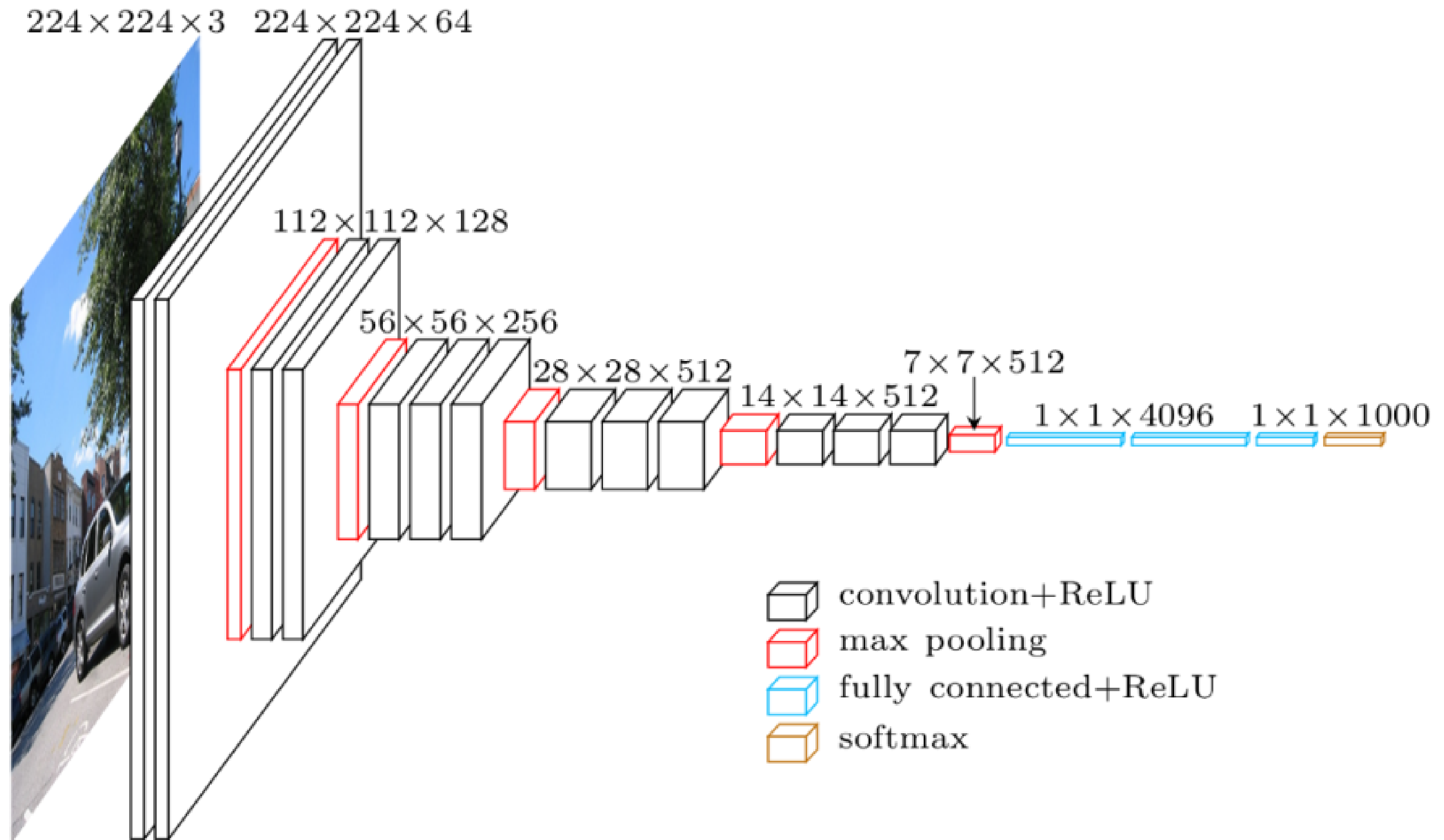
aelion.fr

aelion

AlexNet

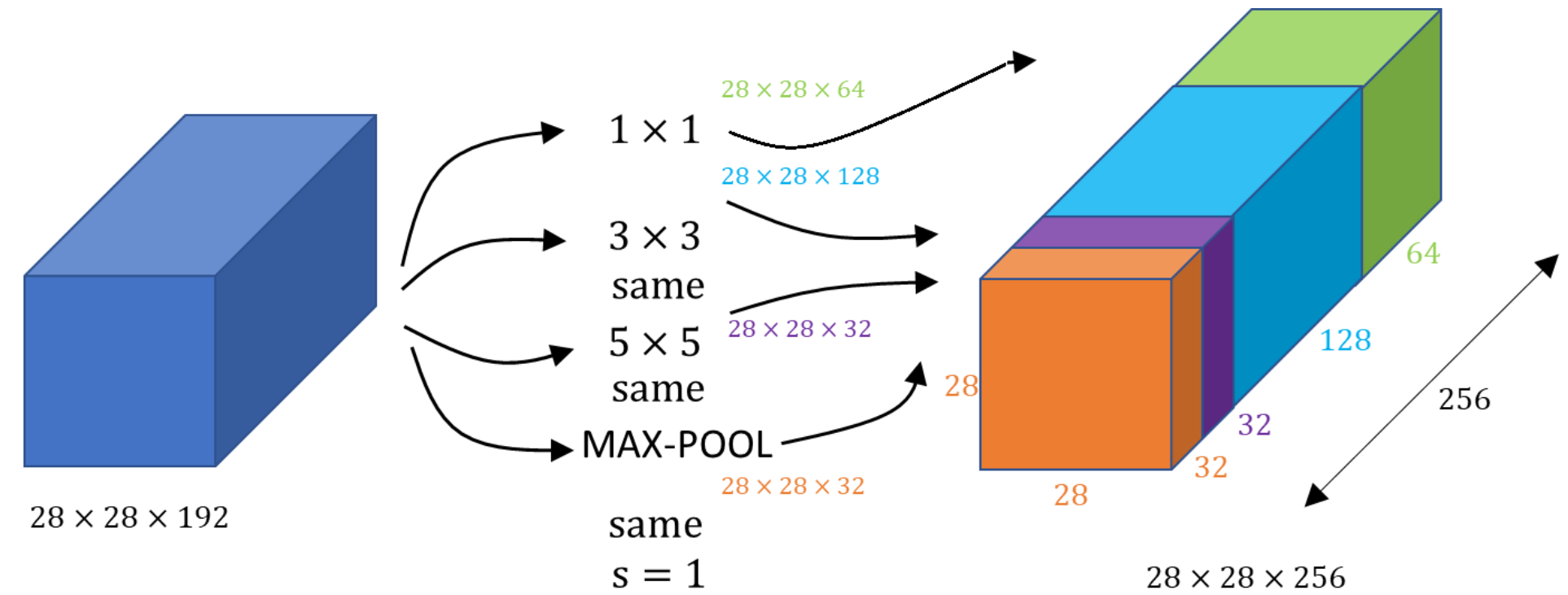


VGG

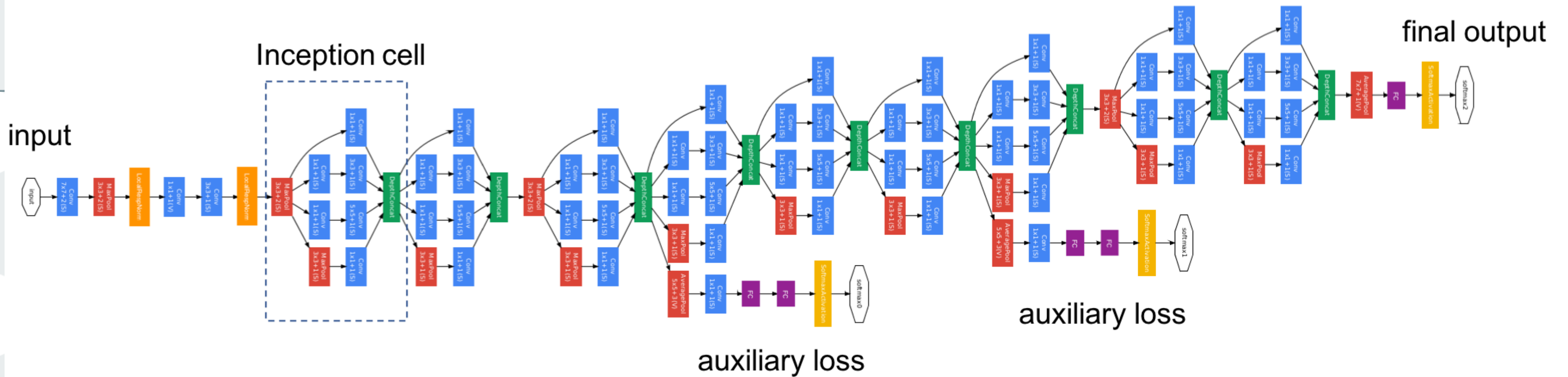


Inception

- Plusieurs tailles de filtres
- Couche de concaténation
- Utilisation de filtres 1×1 pour contrôler la profondeur finale



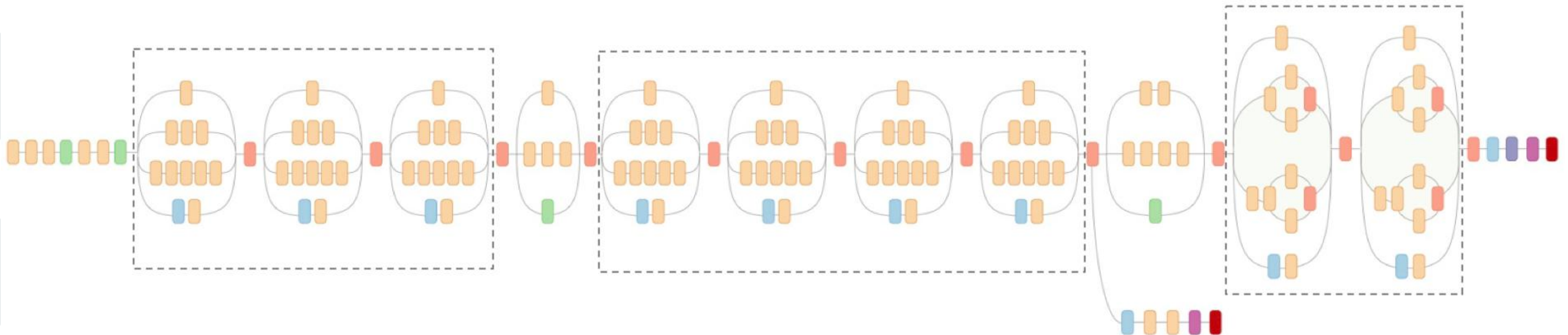
aelien



Architecture Inception v3

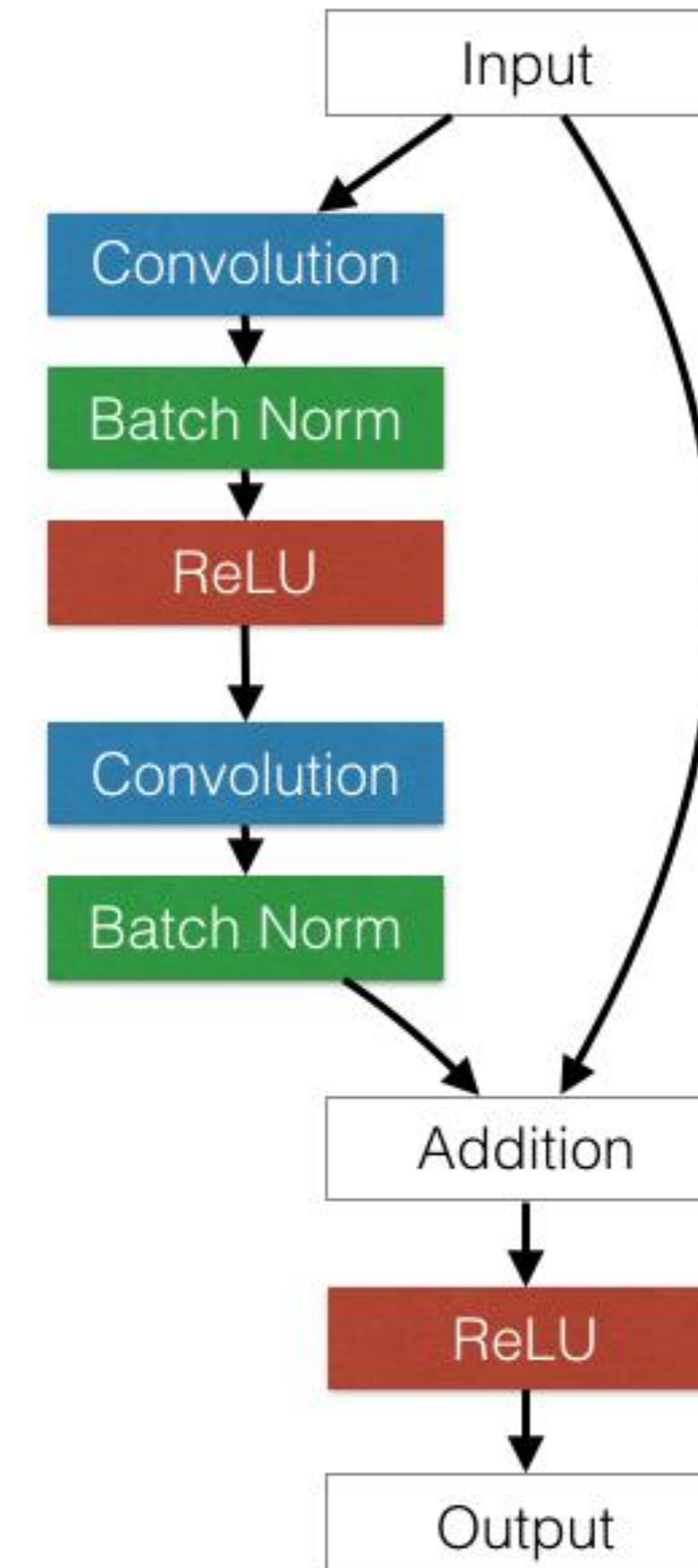
aelion.fr

- 48 couches cachées
- 23 millions de poids à entraîner

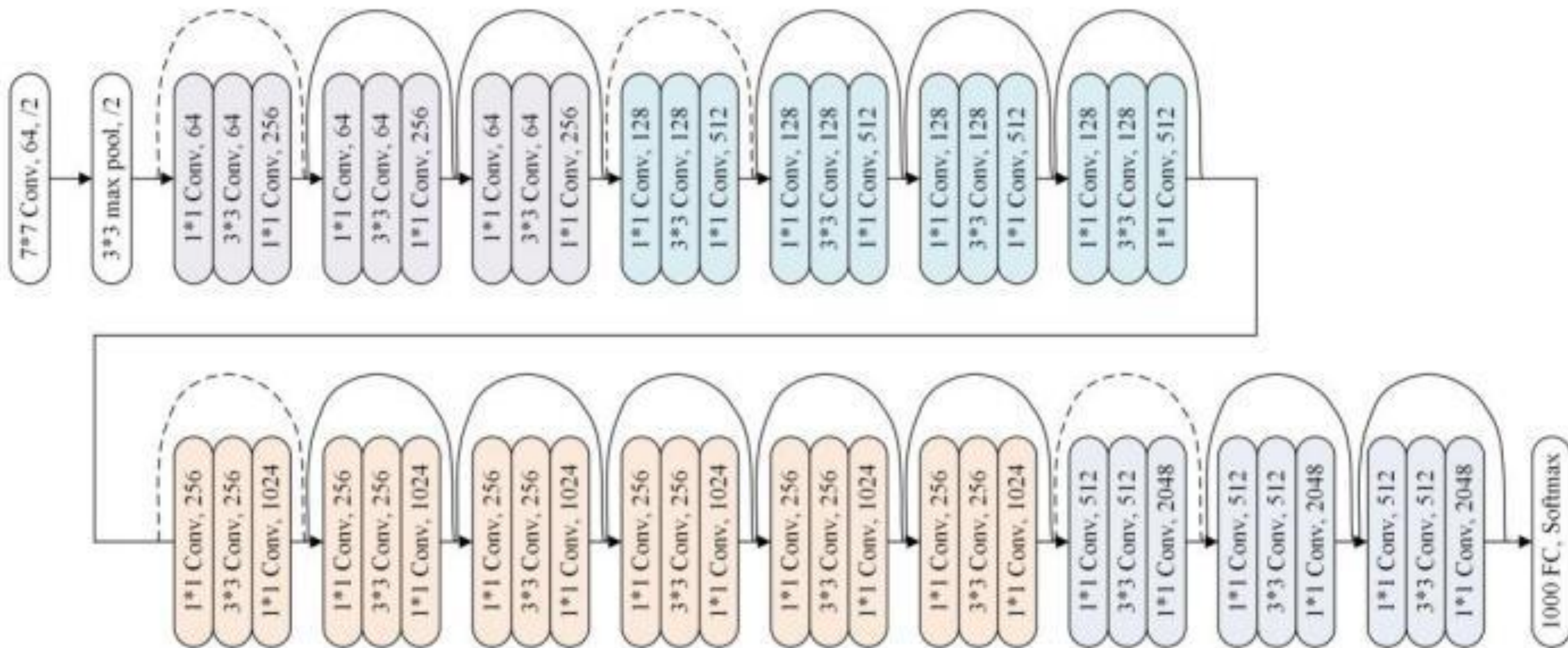


Residual Network (ResNet)

- Ajout de connexions résiduelles « saute-couches »
- Permet d'ajouter de la profondeur à un réseau (plus de couches)
- Si trop de couches, certaines sont « désactivées » par le réseau : elles convergent vers la fonction identité

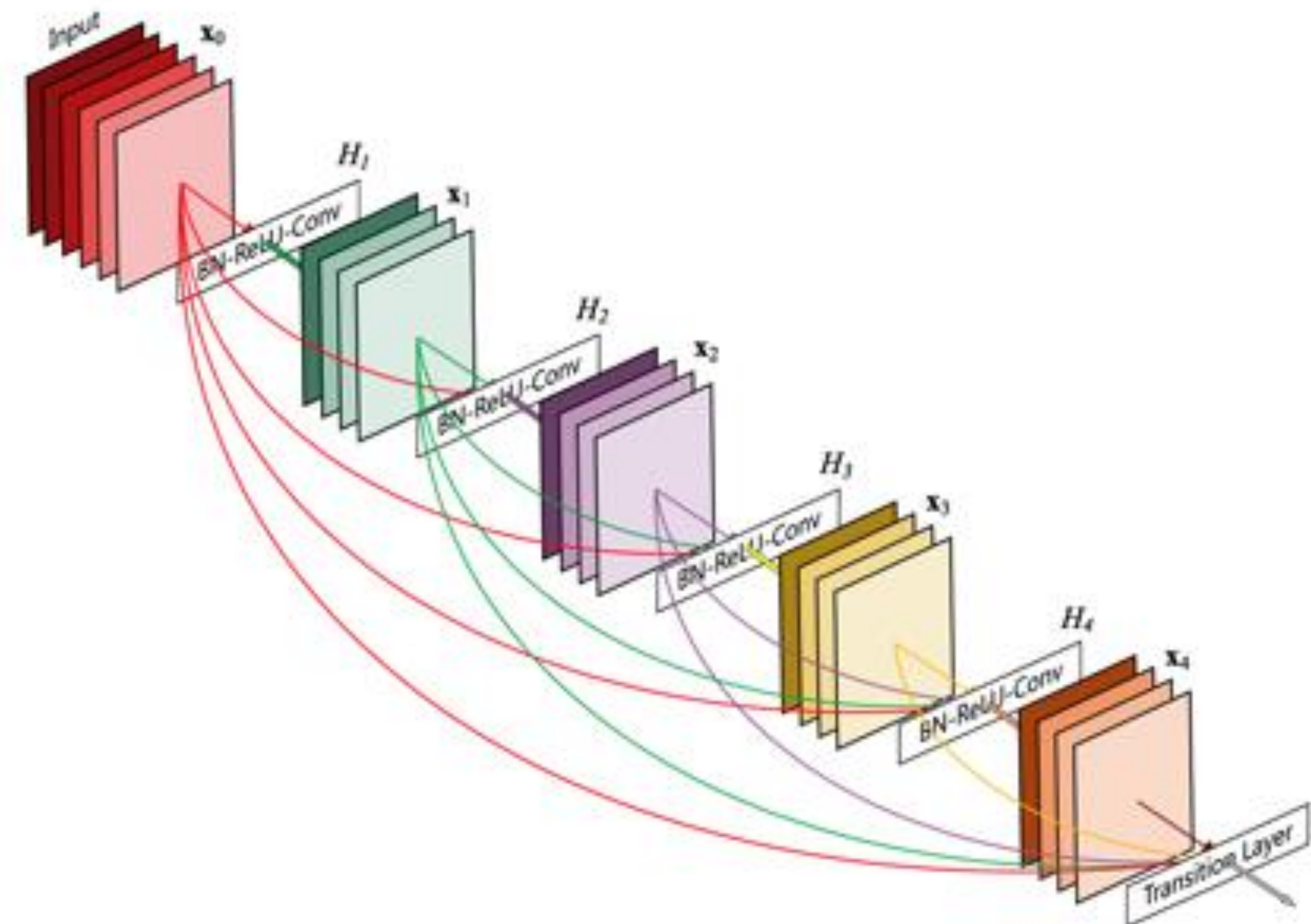


ResNet50



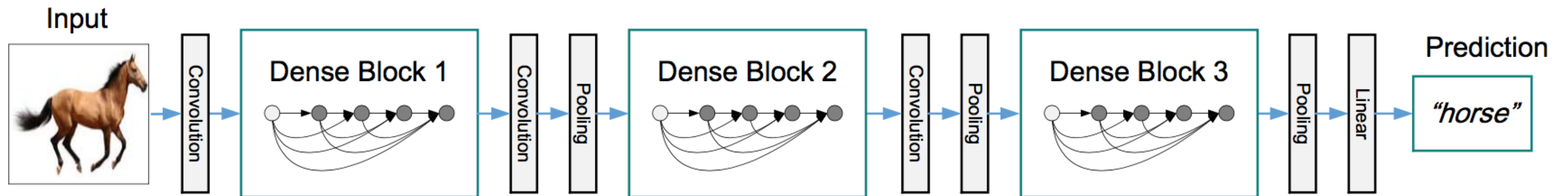
DenseNet

- Toutes les sorties (features) d'une couche sont connectées aux entrées des couches suivantes
- Les features sont largement réutilisées
- **Besoin de moins de filtres par couche**



DenseNet — architecture globale

- Succession de blocs « DenseNet »
- **Meilleures performances pour moins de poids à entraîner**



06

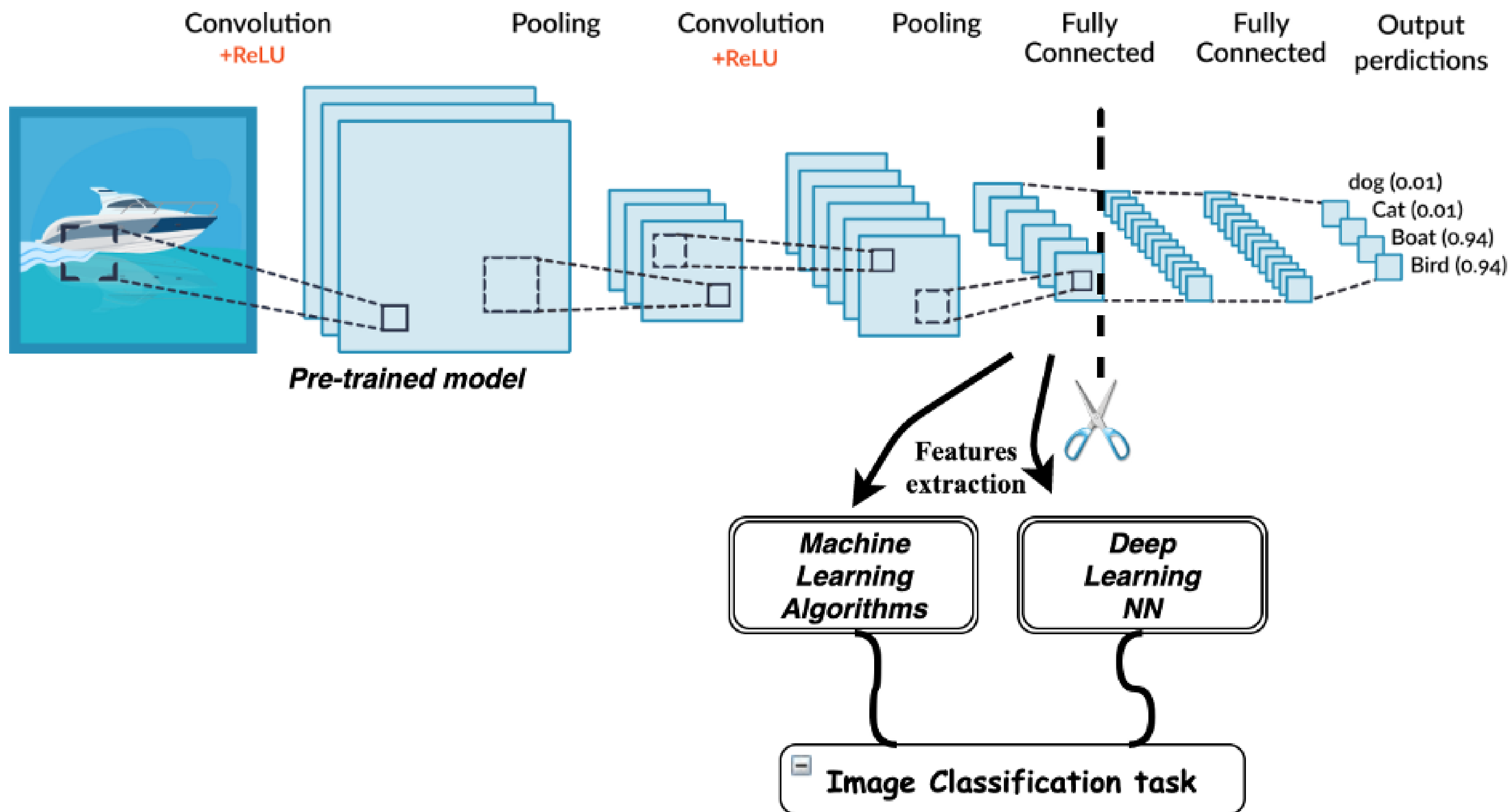
Transfer Learning



Réutiliser des modèles pré-entraînés

Principe

aelion.fr



On réutilise les couches d'un réseau pré-entraîné comme extracteur de features

Le CNN entraîné comme encodeur

- output d'une couche cachée = input d'un nouveau modèle
- nouveau modèle = réseau de neurones ou autre : random forest, boosting...

Le CNN entraîné comme point de départ

- premières couches non entraînaables (« freeze »)
- dernières couches entraînaables
- beaucoup de données = beaucoup de couches entraînaables

Domaines applicables

aelion.fr

- **Images : de nombreux CNN ont été entraînés sur des bases de plusieurs millions d'images labellisées**
- **Sons :**
 - modèles existants en anglais (speech-to-text) : DeepSpeech, WaveNet...
 - en français : pas assez de données — vous pouvez contribuer (Common Voice FR)
 - services cloud payants : API Google, Amazon, Watson...

07

Auto-encodeurs



Apprentissage de représentations

Qu'est-ce qu'un auto-encodeur ?

Réseau de neurones entraîné à reconstruire sa propre entrée en sortie

- Apprentissage non supervisé : aucune étiquette, la cible est l'entrée elle-même

Deux parties symétriques :

- l'encodeur compresse l'entrée vers un espace latent de petite dimension
- le décodeur reconstruit l'entrée à partir de cette représentation compressée

Le goulot d'étranglement force le réseau à n'apprendre que l'information essentielle

Architecture d'un auto-encodeur

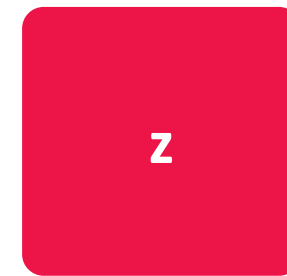
aelion.fr

Entrée x

Sortie $\hat{x}$

Encodeur

Décodeur



Espace latent

(représentation compressée)

Fonction de coût : minimiser l'écart de reconstruction entre l'entrée x et la sortie $\hat{x}$

Cible = entrée : on minimise l'erreur de reconstruction

- Fonction de coût classique :
 - MSE pour des données continues (images, signaux)
 - entropie croisée binaire pour des données dans $[0, 1]$
- Même optimisation que les autres réseaux : descente de gradient + backpropagation (Adam)

La dimension de l'espace latent est l'hyperparamètre clé :

- trop grande : le réseau « recopie » l'entrée sans rien apprendre
- trop petite : reconstruction de mauvaise qualité

Principales variantes

aelion.fr

- **Auto-encodeur débruiteur (denoising)**
 - on bruite l'entrée et on demande de reconstruire l'originale — apprend des représentations robustes
- **Auto-encodeur parcimonieux (sparse)**
 - contrainte de parcimonie sur l'espace latent — features plus interprétables
- **Auto-encodeur convolutif**
 - encodeur/décodeur à base de convolutions — adapté aux images
- **Auto-encodeur variationnel (VAE)**
 - espace latent probabiliste — devient un modèle génératif capable de créer de nouvelles données

- Réduction de dimension : alternative non linéaire à l'ACP (PCA)
- Débruitage d'images et de signaux
- Détection d'anomalies : une erreur de reconstruction élevée signale une donnée atypique
- Pré-entraînement et apprentissage de représentations (lien avec le transfer learning)
- Génération de données nouvelles avec les VAE
- Compression de données



aelion

95 chemin de Gabardie 31200 Toulouse
05 61 49 42 60 - contact@aelion.fr

aelion.fr

